

Re:VIEW テンプレート

TechBooster 編 著

2026-04-11 版 TechBooster 発行

前書き

前書きなのだ。

免責事項

本書に記載された内容は、情報の提供のみを目的としています。したがって、本書を用いた開発、製作、運用は、必ずご自身の責任と判断によって行ってください。これらの情報による開発、製作、運用の結果について、著者はいかなる責任も負いません。

目次

前書き	2
免責事項	2
第 1 章 データベースとは何か	8
1.1 データベースの役割	8
1.1.1 データベースが担う役割	8
1.1.2 データを管理するだけでは十分ではない	9
1.1.3 データベースが提供する機能	9
1.2 ファイルとの違い	10
1.2.1 ファイルは「保存場所」である	10
1.2.2 データベースはデータを管理する	11
1.2.3 データ量が増えると違いはさらに大きくなる	11
1.3 なぜ自作するのか	12
1.3.1 データベースはブラックボックスになりやすい	12
1.3.2 一つずつ機能を作ることによって理解できる	13
1.3.3 本書で目指すこと	13
1.4 本書の進め方	14
1.4.1 「問題」と「改善」を繰り返す	14
1.4.2 第 1 章では最小限のデータベースを作る	14
1.4.3 第 2 章では永続化を実装する	15
1.4.4 その後も段階的に発展させる	16
1.4.5 完成形を目指すのではなく、変化を理解する	16
1.4.6 この節のまとめ	16
1.5 本書で作るデータベースの全体像	17
1.5.1 データベース全体の構成	17
1.5.2 本書を通して学べること	18

目次

1.5.3	序章のまとめ	18
第 2 章	最小のデータベース	19
2.1	対話型プログラム (REPL) の作成	19
2.1.1	最初のデータベース	19
2.1.2	REPL の実装	20
2.2	入力とコマンドの扱い	21
2.2.1	コマンドの分割	21
2.2.2	コマンド名を取得する	22
2.2.3	コマンドに応じた処理の分岐	22
2.3	CRUD 操作の実装	23
2.3.1	insert — データを登録する	24
2.3.2	select — データを取得する	24
2.3.3	update — データを更新する	25
2.3.4	delete — データを削除する	26
2.3.5	動作確認	26
2.4	Map によるデータ管理	27
2.4.1	Map とは	27
2.4.2	なぜ HashMap を使うのか	28
2.5	1.5 最小 DB の動作確認	29
2.5.1	データを登録する	29
2.5.2	データを取得する	30
2.5.3	データを更新する	30
2.5.4	データを削除する	31
2.5.5	エラー入力の確認	32
2.5.6	現在のデータベースの問題点	32
2.5.7	この章のまとめ	33
第 3 章	データの永続化	34
3.1	2.1 なぜ永続化が必要なのか	34
3.1.1	メモリ上のデータは消えてしまう	35
3.1.2	実際のデータベースではどうしているのか	35
3.1.3	ファイルへ保存する	36
3.1.4	第 2 章で実装すること	37
3.2	2.2 ファイルへの保存	37

3.2.1	保存先を決める	37
3.2.2	Path によるファイル管理	38
3.2.3	保存先ディレクトリの作成	38
3.2.4	ファイルが存在する場合	39
3.2.5	ファイル保存の流れ	39
3.2.6	この節のまとめ	40
3.3	2.3 データ形式の設計	40
3.3.1	保存形式を考える	40
3.3.2	key,value 形式で保存する	41
3.3.3	保存時の変換	41
3.3.4	読み込み時の変換	42
3.3.5	不正なデータへの対応	44
3.3.6	この節のまとめ	44
3.4	2.4 更新処理の実装	45
3.4.1	保存処理の流れ	45
3.4.2	レコードを文字列へ変換する	46
3.4.3	ファイルへ書き込む	47
3.4.4	CRUD 操作と保存処理	47
3.4.5	更新のたびに保存する理由	48
3.4.6	この実装の課題	49
3.4.7	この節のまとめ	50
3.5	2.5 永続化の確認	50
3.5.1	データを登録する	50
3.5.2	データベースファイルを確認する	51
3.5.3	プログラムを再起動する	51
3.5.4	更新・削除も保存される	52
3.5.5	第 1 章との違い	53
3.5.6	現在の実装の課題	53
3.5.7	この章のまとめ	54
3.6	2.6 第 2 章のまとめと課題	55
3.6.1	第 2 章で実装した内容	55
3.6.2	第 1 章との比較	55
3.6.3	新たに見えてきた課題	56
3.6.4	なぜ非効率なのか	57
3.6.5	次章へ	57

目次

第 4 章	ページ (Page) によるデータ配置の改善	58
4.1	第 2 章までの課題：ファイル全体の書き直し	58
4.2	ページという単位の導入	59
4.3	固定長スロットによるレコードの設計	59
4.4	ページ単位での読み書きの実装	61
4.4.1	挿入 (insert)	61
4.4.2	検索 (select)	63
4.4.3	更新 (update)	65
4.4.4	削除 (delete)	66
4.4.5	処理時間の計測	66
4.5	まとめと次章への課題	67
第 5 章	B-Tree による検索の高速化と領域管理	68
5.1	第 4 章までの課題：線形探索の限界	68
5.2	検索を高速化するデータ構造の導入	68
5.3	B-Tree の実装：RecordId の導入	69
5.3.1	データベースが B-Tree を選ぶ最大の理由	70
5.3.2	ノードの中身と全体構造のイメージ	71
5.4	B-Tree の実装	72
5.4.1	ノードの表現 (BTreeNode) と最小次数	72
5.4.2	検索処理 (Search)	74
5.4.3	挿入とノードの分割 (Split)	74
5.4.4	削除処理におけるバランスの維持 (Borrow と Merge)	76
5.5	起動時の B-Tree 構築と空き領域管理	78
5.5.1	B-Tree の再構築 (buildTree)	79
5.5.2	空き領域リスト (freeList) による挿入の高速化	79
5.6	B-Tree を用いた CRUD の高速化	80
5.6.1	検索処理 (select) の改善	80
5.6.2	挿入処理 (insert) の改善	81
5.6.3	更新 (update) と削除 (delete) の改善	82
5.6.4	実行速度の比較：どれくらい速くなったのか	82
5.7	まとめと次章への課題	83
第 6 章	テーブルが定義され、データが保存されるまで	85
6.1	Schema は後続の処理で繰り返し使われる	89

6.2	SELECT では Schema を使ってレコードを読み戻す	95
6.3	UPDATE では RecordId を使って同じ位置を書き換える	97
6.4	DELETE ではレコードを物理的に消去しない	98
6.5	chapter05 におけるテーブルのライフサイクル	100
第 7 章	章タイトル	103
7.1	節のキャプション	103
第 8 章	章タイトル	105
8.1	節のキャプション	105
著者紹介		107

第 1 章

データベースとは何か

1.1 データベースの役割

私たちは普段、さまざまなデータを扱っています。例えば、スマートフォンの連絡先には名前や電話番号が保存され、SNS には投稿やコメントが蓄積されています。EC サイトでは商品情報や注文履歴が管理され、銀行では口座情報や取引履歴が記録されています。これらの情報は、一時的に利用されるものではなく、長期間にわたって保存され、必要なときに素早く取り出せる必要があります。このような大量のデータを整理・管理するための仕組みがデータベース（Database）です。

1.1.1 データベースが担う役割

データベースには、単にデータを保存するだけでなく、さまざまな役割があります。例えば、顧客情報を管理するシステムでは、次のような操作が必要になります。

- 新しい顧客を登録する
- 登録済みの顧客情報を検索する
- 住所や電話番号を更新する
- 退会した顧客を削除する

これらはそれぞれ、

- 登録（Create）
- 検索（Read）
- 更新（Update）
- 削除（Delete）

1.1 データベースの役割

という 4 つの基本操作に分類できます。この 4 つは **CRUD**（**Create, Read, Update, Delete**）と呼ばれ、多くのデータベースで基本となる操作です。本書でも、まずはこの 4 つの操作を実装するところから始めます。

1.1.2 データを管理するだけでは十分ではない

データを保存するだけであれば、Java の変数やコレクションを利用することもできます。例えば、Java の ‘HashMap’ を使えば、

```
1 → Alice  
2 → Bob  
3 → Carol
```

のように、ID と名前を対応付けて管理できます。この方法なら、データの登録や検索も簡単に実装できます。しかし、この方法には大きな問題があります。プログラムを終了すると、メモリ上のデータはすべて失われてしまうのです。また、データが数件であれば問題ありませんが、数万件、数百万件と増えていくと、検索や更新を効率よく行うための工夫も必要になります。つまり、「データを保存する」だけでは、実用的なデータ管理システムとは言えません。

1.1.3 データベースが提供する機能

実際のデータベースは、こうした問題を解決するために、さまざまな機能を備えています。例えば、

- プログラムを終了してもデータが失われないように保存する
- 大量のデータを高速に検索する
- データを効率よく更新する
- 統一された問い合わせ方法を提供する

といった機能があります。私たちが普段利用しているデータベース管理システム（DBMS）は、これらを意識することなく利用できるよう設計されています。しかし、その内部では多くの工夫が行われています。

第1章 データベースとは何か

1.2 ファイルとの違い

前節では、データベースはデータを保存し、必要なときに素早く取り出すための仕組みであることを説明しました。では、データを保存するだけであれば、データベースを使う必要はあるのでしょうか。例えば、Java ではファイルへ文字列を書き込むだけでデータを保存できます。

```
1,Alice  
2,Bob  
3,Carol
```

このようなテキストファイルを作成すれば、プログラムを終了したあとでもデータは残ります。一見すると、これだけでも十分なように思えます。しかし、実際にはファイルだけでは解決できない問題が数多くあります。

1.2.1 ファイルは「保存場所」である

ファイルの役割は、データをディスク上へ保存することです。例えば、顧客情報を保存する場合、

```
1,Alice  
2,Bob  
3,Carol
```

という内容をファイルへ書き込めば、電源を切ってもデータは失われません。つまり、ファイルはデータを残しておくための仕組みです。一方で、ファイル自身は、

- 顧客情報なのか
- 商品情報なのか
- 注文履歴なのか

といった意味を理解しているわけではありません。また、「ID が 2 のデータを探したい」「名前を更新したい」といった操作も提供していません。ファイルは、あくまでデータを保存するための入れ物に過ぎないのです。

1.2.2 データベースはデータを管理する

これに対して、データベースはデータを保存するだけではありません。例えば、

ID = 2

という条件で検索すると、

Bob

というデータだけを取り出すことができます。また、

Bob

を

Robert

へ変更したり、不要になったデータを削除したりすることもできます。つまり、データベースは

- 保存する
- 検索する
- 更新する
- 削除する

という一連の操作を統一的に提供します。これが、単なるファイルとの大きな違いです。

1.2.3 データ量が増えると違いはさらに大きくなる

数件のデータであれば、ファイルを開いて目的の行を探すことも難しくありません。しかし、例えば 100 万件のデータが保存されていたらどうでしょうか。目的のデータを探すためには、先頭から 1 行ずつ確認していかなければならないかもしれません。さらに、途中のデータを書き換えると、その後ろのデータの位置も変わるため、ファイル全体を書き

第1章 データベースとは何か

直す必要が生じる場合があります。データ量が増えるほど、このような処理は非効率になります。データベースは、この問題を解決するために、

- データを効率よく配置する
- 必要なデータだけを読み込む
- 高速に検索できる索引（インデックス）を持つ

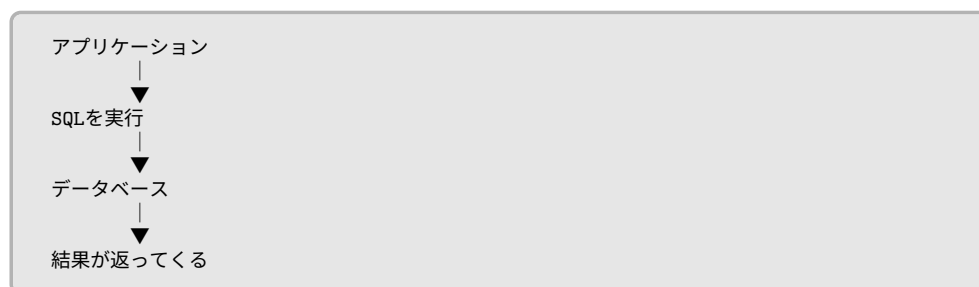
といった工夫を取り入れています。

1.3 なぜ自作するのか

前節では、ファイルとデータベースの違いについて説明しました。現在では、高性能なデータベース管理システム（DBMS）が数多く公開されており、多くの開発現場ではそれらを利用してシステムを構築します。そのため、実際の開発では、データベースを一から実装する機会はほとんどありません。では、なぜ本書では、あえてデータベースを自作するのでしょうか。

1.3.1 データベースはブラックボックスになりやすい

多くの開発者は、データベースを次のように利用しています。



例えば、

```
SELECT * FROM users WHERE id = 1;
```

という SQL を実行すれば、目的のデータが返ってきます。しかし、

- SQL はどのように解析されるのか
- データはどこに保存されているのか

1.3 なぜ自作するのか

- どのように検索されているのか
- なぜ高速に検索できるのか

といった内部の仕組みを意識することはほとんどありません。もちろん、通常のアプリケーション開発では、それでも十分です。一方で、データベースの性能や設計を理解したい場合には、「内部で何が起きているのか」を知ることが重要になります。

1.3.2 一つずつ機能を作ることで理解できる

データベースは、多くの技術を組み合わせて実現されています。例えば、

- データを保存する
- データを検索する
- データを更新する
- データをファイルへ保存する
- 高速に検索する
- SQL を解析する

といった機能が組み合わさって、普段利用しているデータベースが動作しています。しかし、最初からこれらすべてを理解しようとする、仕組みが複雑で全体像を把握することが難しくなります。そこで本書では、最小限のデータベースから始め、機能の一つずつ追加していきます。一つひとつの改善点とその解決方法を追いながら実装することで、「なぜこの仕組みが必要なのか」を理解しやすくなります。

1.3.3 本書で目指すこと

本書の目的は、商用データベースと同じ機能を持つソフトウェアを作ることではありません。実際のデータベースには、トランザクション制御、障害復旧、同時実行制御、分散処理など、多くの高度な機能があります。これらをすべて実装するには、大規模なコードと専門的な知識が必要になります。そこで本書では、データベースの基本となる仕組みに焦点を当てます。実際にコードを書きながら、

- データはどのように管理されるのか
- なぜ永続化が必要なのか
- なぜインデックスが高速なのか
- SQL はどのように実行されるのか

といった疑問を、一つずつ解決していきます。完成するデータベースはシンプルです

第1章 データベースとは何か

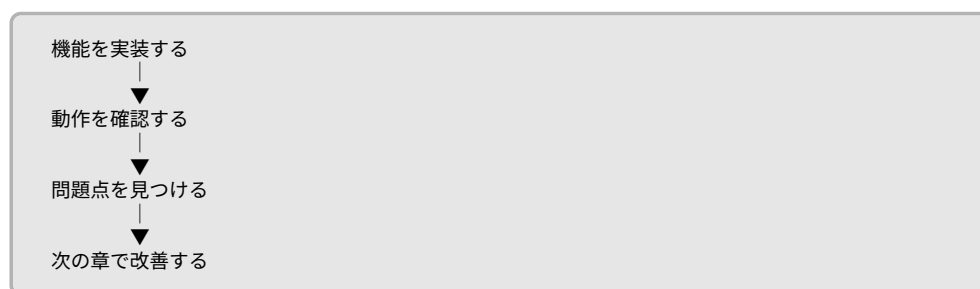
が、その中には実際のデータベースでも使われている基本的な考え方が数多く含まれています。

1.4 本書の進め方

前節では、本書がデータベースを一から実装しながら、その内部構造を理解することを目的としていることを説明しました。しかし、データベースは多くの技術を組み合わせて作られているため、最初から完成形を目指す、どの機能が何のために存在するのかが分かりにくくなってしまいます。そこで本書では、**最小限のデータベースを出発点とし、問題点の一つずつ解決しながら機能を追加していく**という方法で学習を進めます。

1.4.1 「問題」と「改善」を繰り返す

本書では、各章で新しい機能を実装すると同時に、その実装が抱える課題についても確認します。そして、その課題を次の章で改善することで、データベースを少しずつ発展させていきます。つまり、本書全体は次のような流れで進みます。



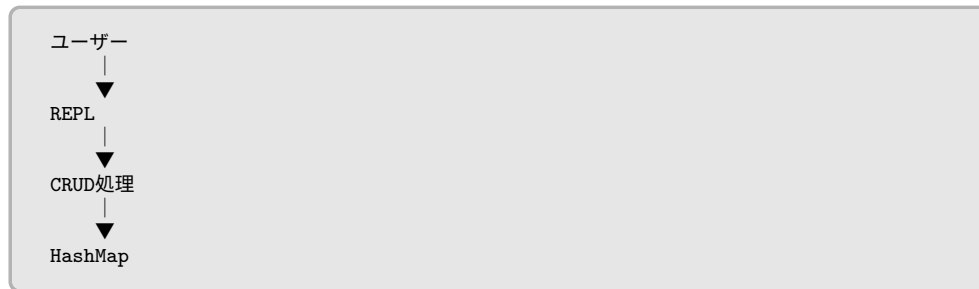
この流れを繰り返すことで、「なぜ新しい仕組みが必要になるのか」を自然な形で理解できます。

1.4.2 第1章では最小限のデータベースを作る

最初にするデータベースは、非常にシンプルです。Java の ‘HashMap’ を利用し、

- データを登録する
- データを取得する
- データを更新する
- データを削除する

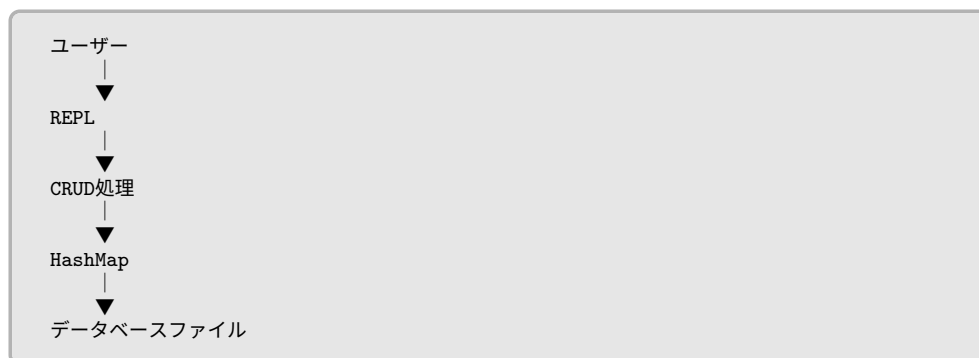
という CRUD 操作だけを実装します。構成は次のようになります。



この時点では、データベースとして最低限の機能を持っています。しかし、データはメモリ上にしか存在しないため、プログラムを終了するとすべて失われてしまいます。これが、第 1 章で見つかる最初の課題です。

1.4.3 第 2 章では永続化を実装する

第 2 章では、第 1 章で見つかった問題を解決します。‘HashMap’ に保存していたデータをファイルへ書き出し、起動時に再び読み込むことで、データを永続的に保持できるようにします。構成は次のように変化します。



これにより、プログラムを終了してもデータが失われなくなります。一方で、更新のたびにファイル全体を書き直しているため、データ量が増えると処理が遅くなるという新たな課題も見えてきます。

第 1 章 データベースとは何か

1.4.4 その後も段階的に発展させる

第 3 章以降も同じ流れで進みます。例えば、

- 第 3 章では、ファイル全体を書き直す問題を改善するためにページ単位でデータを管理します。
- 第 4 章では、全件検索の問題を解決するためにインデックスを導入します。
- 第 5 章では、より使いやすい操作方法として SQL を解析できるようにします。
- 第 6 章以降では、SQL を効率よく実行するための仕組みや、データベース全体の構造を整理していきます。

このように、**前の章で見つかった課題が、次の章で新しい技術を導入する理由**になります。

1.4.5 完成形を目指すのではなく、変化を理解する

本書では、各章ごとにデータベースの設計が少しずつ変化します。そのため、同じ処理でも実装方法が変わることがあります。例えば、第 1 章ではデータを ‘HashMap’ に保存しますが、第 2 章ではファイルへの保存処理が追加されます。さらに、第 3 章ではページ単位のデータ管理へと変更されます。これは以前の実装が間違っていたからではありません。その時点で必要な機能を最も分かりやすく実現する方法を採用し、新しい課題が見つかった段階で改善していくためです。実際のソフトウェア開発でも、一度に完成形を作ることはほとんどありません。まずはシンプルに実装し、必要に応じて設計を見直しながら発展させていきます。本書では、その過程そのものを体験できるよう構成しています。

1.4.6 この節のまとめ

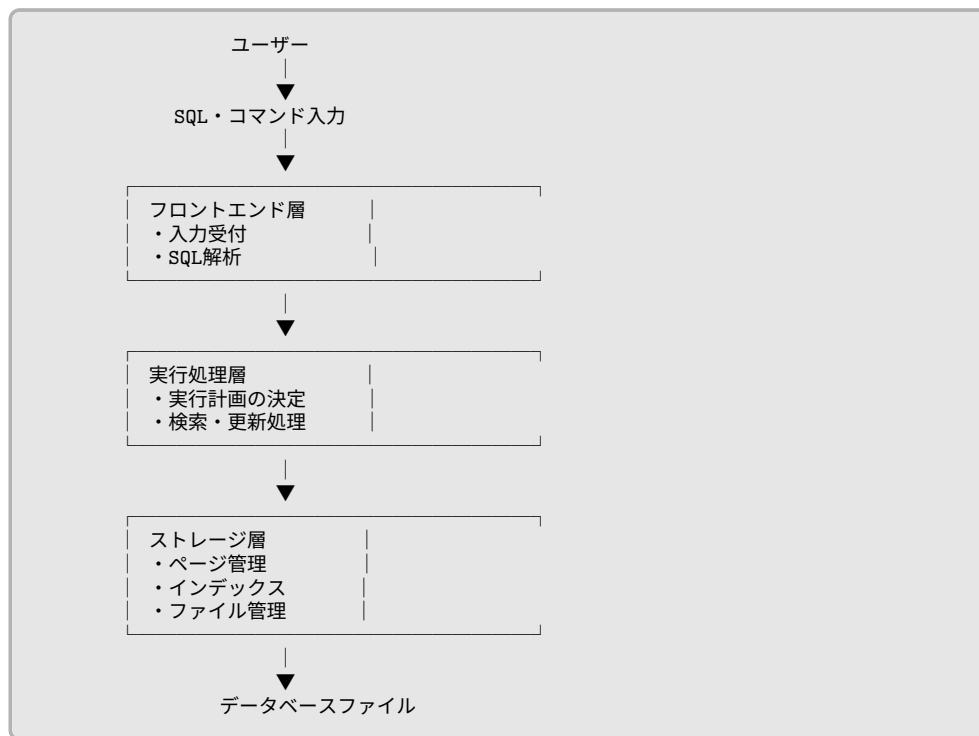
本書では、最初から完成したデータベースを実装するのではなく、「実装 → 動作確認 → 問題発見 → 改善」という流れを繰り返しながら、データベースを段階的に発展させていきます。第 1 章では、‘HashMap’ を利用した最小限のデータベースを作成します。第 2 章ではファイルへの保存機能を追加し、その後も各章で新たな課題を解決しながら、より実用的なデータベースへと成長させていきます。次節では、本書を通して完成するデータベース全体の構成を概観し、各章で追加される機能がどのようにつながっていくのかを確認します。

1.5 本書で作るデータベースの全体像

前節では、本書が最小限のデータベースから始め、問題点を一つずつ解決しながら機能を追加していく構成であることを説明しました。最後に、本書を読み終えたときに完成するデータベースが、どのような構成になっているのかを見ておきましょう。

1.5.1 データベース全体の構成

本書で実装するデータベースは、大きく分けると次の3つの役割で構成されます。



最終的には、このような層構造を持つデータベースを完成させます。各層が役割を分担することで、機能を追加しやすく、保守しやすい構成になります。

第1章 データベースとは何か

1.5.2 本書を通して学べること

本書を読み終える頃には、データベースを利用するだけでは見えにくい、内部の仕組みを理解できるようになります。例えば、

- データはどのように保存されるのか
- なぜデータベースは高速に検索できるのか
- SQL はどのように解析されるのか
- 実行方法はどのように決定されるのか
- データベース内部はどのような層構造になっているのか

といった疑問に、自分で実装したコードを通して答えられるようになることを目指します。本書で扱う実装は教育用に簡略化していますが、データベースの基本的な考え方は、実際のデータベース管理システムにも共通しています。

1.5.3 序章のまとめ

本章では、データベースの役割やファイルとの違い、自作する目的、そして本書の進め方について説明しました。本書では、最初から完成されたデータベースを扱うのではなく、最小限の実装から始め、課題を見つけながら段階的に改善していきます。次章からは、いよいよ実際にコードを書き始めます。まずは、**Java の ‘HashMap’ を利用した最小限のデータベース**を実装し、データベースの基本となる CRUD 操作と対話型プログラム (REPL) の仕組みを構築していきます。ここから、本書全体を通したデータベース開発の第一歩が始まります。

第2章

最小のデータベース

2.1 対話型プログラム (REPL) の作成

データベースは、一度起動したらすぐに終了するプログラムではありません。ユーザーからの入力を待ち続け、入力された命令を実行し、その結果を表示することを繰り返します。例えば、多くのデータベースでは次のような対話形式で操作を行います。

```
> SELECT * FROM users;  
1|Alice|20  
2|Bob|18
```

このようなプログラムは **REPL (Read-Eval-Print Loop)** と呼ばれます。REPL とは、

- **Read** : ユーザーから入力を受け取る
- **Eval** : 入力された命令を実行する
- **Print** : 実行結果を表示する
- **Loop** : これらを繰り返す

という4つの処理から構成されます。本書で作成するデータベースも、この REPL 形式を採用します。

2.1.1 最初のデータベース

まずはデータベース本体となる 'nekoDB' クラスを作成します。コンストラクタでは、データを保持するための 'HashMap'、キーボード入力を受け取る 'Scanner'、入力を解析するための 'SimpleParser' を初期化します。

第2章 最小のデータベース

```
public nekoDB() {
    db = new HashMap<>();
    scanner = new Scanner(System.in);
    parser = new SimpleParser();
    System.out.println("Welcome to nekoDB!");
}
```

現時点では、‘HashMap’ がデータベースそのものになります。プログラムを起動すると、

```
Welcome to nekoDB!
```

というメッセージが表示され、データベースの開始を知らせます。

2.1.2 REPLの実装

データベースが起動したら、ユーザーが終了を指示するまで入力を受け付け続けます。そのために、‘start()’ メソッドでは無限ループを用意しています。

```
while (true) {
    System.out.print("db > ");

    if (!scanner.hasNextLine()) {
        break;
    }

    String[] tokens = parser.parse(scanner.nextLine());

    // コマンドを実行
}
```

このコードを実行すると、画面には次のように表示されます。

```
db >
```

ここでユーザーはコマンドを入力します。入力された文字列は1行分読み込まれ、

```
parser.parse(...)
```

によって入力を解析し、後続の処理へ渡します。

2.2 入力とコマンドの扱い

ユーザーからの入力を受け付ける REPL を作成しましたが、この時点では入力された文字列は単なる文字列であり、プログラムはその意味を理解できません。例えば、

```
db > insert 1 Alice
```

という入力があったとしても、プログラムにとっては

```
"insert 1 Alice"
```

という一つの文字列に過ぎません。データベースとして動作させるには、この文字列を

- 実行するコマンド
- 操作対象のデータ

に分けて解釈する必要があります。本書では、この役割を **SimpleParser** クラスが担当します。

2.2.1 コマンドの分割

最初に、入力された文字列を空白で分割します。例えば、

```
insert 1 Alice
```

という入力は、

```
["insert", "1", "Alice"]
```

という 3 つの要素に分割されます。この処理は `SimpleParser` クラスの `parse()` メソッドで行っています。

第2章 最小のデータベース

```
public String[] parse(String sql) {  
    String[] tokens = sql.trim().split("\\s+");  
    tokens[0] = tokens[0].toLowerCase();  
    return tokens;  
}
```

入力の解析結果は次のようになり、コマンドと引数を個別に扱えるようになります。

入力	分割結果
`insert 1 Alice`	`["insert","1","Alice"]`
`select`	`["select"]`
`delete 5`	`["delete","5"]`

2.2.2 コマンド名を取得する

データベースが最初に知りたいのは、「どの操作を実行するのか」です。そこで、分割した配列の最初の要素をコマンドとして取り出します。

```
public String getCommand(String[] tokens) {  
    return tokens.length > 0 ? tokens[0] : "";  
}
```

例えば、

```
["insert", "1", "Alice"]
```

であれば、

```
insert
```

が返されます。これにより、以降の処理では入力全体を調べる必要がなくなり、コマンド名だけを見て処理を分岐できるようになります。

2.2.3 コマンドに応じた処理の分岐

‘start()’メソッドでは、取得したコマンド名をもとに処理を切り替えています。

2.3 CRUD 操作の実装

```
String[] tokens = parser.parse(scanner.nextLine());
String command = parser.getCommand(tokens);
```

取得したコマンドに対して、

```
if (command.equals("insert") && tokens.length == 3) {
    insert(tokens[1], tokens[2]);
}
```

のように判定を行います。例えば、

```
insert 1 Alice
```

と入力すると、

```
command = "insert"
tokens[1] = "1"
tokens[2] = "Alice"
```

となり、

```
insert("1", "Alice");
```

が呼び出されます。このように、入力された文字列を解析し、適切なメソッドへ処理を振り分けることで、データベースはさまざまな命令を実行できるようになります。

2.3 CRUD 操作の実装

ユーザーが入力したコマンドを解析し、実行する処理を決定できるようになりましたが、この時点ではデータベースとしての機能はまだありません。そこで本節では、データベースの基本操作である **CRUD** を実装します。CRUD とは、データを扱うための 4 つの基本操作の頭文字を取ったものです。

第2章 最小のデータベース

操作	コマンド	説明
Create	<code>`insert`</code>	データを登録する
Read	<code>`select`</code>	データを取得する
Update	<code>`update`</code>	データを更新する
Delete	<code>`delete`</code>	データを削除する

多くのデータベースは、これら4つの操作を基本として構成されています。本書でも、まずはこの最小限の機能を実装していきます。

2.3.1 insert — データを登録する

新しいデータを登録するには、`'insert'` コマンドを使用します。例えば、

```
db > insert 1 Alice
```

と入力すると、

```
キー: 1  
値: Alice
```

というデータをデータベースへ登録します。REPL では、コマンド名と引数の数を確認し、`'insert()'` メソッドを呼び出します。`insert()` ではキーの重複チェックを行い、`'HashMap'`にキーと値の組み合わせを追加します。

```
if (db.putIfAbsent(id, value) != null) {  
    System.out.println("Key already exists.");  
    return;  
}
```

2.3.2 select — データを取得する

登録したデータを確認するには、`'select'` コマンドを使用します。`'select'` には2つの使い方があります。

1つ目の使い方は、キーを指定せず、登録されているすべてのデータを取得します。

2.3 CRUD 操作の実装

```
db > select
(1,Alice)
(2,Bob)
```

実装では、‘HashMap’ のすべての要素を順番に取り出して表示します。

```
db.forEach((id, name) ->
    System.out.println("(" + id + ", " + name + ")");
```

2 通りの使い方は、キーを指定し、そのデータだけを取得します。

```
db > select 2
Bob
```

実装では、指定されたキーの存在を確認し、‘HashMap’から取り出して表示します。

```
if (!db.containsKey(id)) {
    System.out.println("Record not found.");
} else {
    System.out.println(db.get(id));
}
```

2.3.3 update — データを更新する

登録済みのデータを変更するには、‘update’ コマンドを使用します。例えば、

```
db > update 2 Robert
```

を実行すると、ID=2 に対応する値が

```
Bob
```

から

第2章 最小のデータベース

```
Robert
```

へ変更されます。‘update()’メソッドでは、まず対象のキーが存在するかを確認します。

```
if (!db.containsKey(id)) {  
    System.out.println("Record not found.");  
    return;  
}
```

存在する場合は、

```
db.put(id, value);
```

によって値を上書きします。

2.3.4 delete — データを削除する

不要になったデータは、‘delete’ コマンドで削除します。例えば、

```
db > delete 1
```

を実行すると、ID=1 のレコードがデータベースから削除されます。‘delete()’では、‘update()’メソッドと同様に、まず対象のキーが存在するか確認します。存在する場合は、

```
db.remove(id);
```

によってレコードを削除します。

2.3.5 動作確認

ここまで実装すると、データベースは CRUD 操作を一通り実行できるようになります。

```
Welcome to nekoDB!  
  
db > insert 1 Alice
```

```
Inserted.

db > insert 2 Bob
Inserted.

db > select
(1,Alice)
(2,Bob)

db > update 2 Robert
Updated.

db > select 2
Robert

db > delete 1
Deleted.

db > select
(2,Robert)
```

登録したデータを取得し、更新し、不要になれば削除できることが確認できます。

2.4 Map によるデータ管理

前節では、‘insert’、‘select’、‘update’、‘delete’ の 4 つの基本操作を実装しました。これらの処理では、データを保存するための共通の仕組みとして **Map** を利用しています。本節では、Map がどのようなデータ構造であり、データベースの内部でどのような役割を果たしているのかを見ていきます。

2.4.1 Map とは

Java の ‘Map’ は、「キー」と「値」の組を管理するデータ構造です。例えば、

キー(ID)	値(Name)
1	Alice
2	Bob
3	Carol

というデータは、Map では次のように表現できます。

第2章 最小のデータベース

```
1 → Alice  
2 → Bob  
3 → Carol
```

本書では、このキーをレコードの ID、値をレコードの内容として扱います。プログラムでは、データベース本体を次のように宣言しています。

```
private Map<Integer, String> db;
```

そして、コンストラクタで

```
db = new HashMap<>();
```

を実行することで、空のデータベースを作成しています。

2.4.2 なぜ HashMap を使うのか

Map にはさまざまな実装がありますが、本書では ‘HashMap’ を採用しています。その理由は、指定したキーに対応するデータを高速に検索できるためです。例えば、

```
db > select 2
```

を実行すると、

```
db.get(2);
```

によって ID が 2 のデータを取得できます。データ数が増えても、キーを指定した検索は効率的に行えるため、小規模なデータベースの実装には適しています。なお、本章では Map の内部構造までは扱いません。重要なのは、「キーを指定すると対応する値を取得できる」という特徴です。

そして、前節で実装した 4 つの操作は、すべて Map の基本操作に対応しています。

データベース操作	Mapの操作	
insert	<code>putIfAbsent()</code>	
select	<code>get()</code>	
update	<code>put()</code>	
delete	<code>remove()</code>	

2.5 1.5 最小 DB の動作確認

ここまでで、データベースとして最低限の機能が完成しました。実装した機能は次の 4 つです。

コマンド	機能
<code>insert</code>	データを登録する
<code>select</code>	データを取得する
<code>update</code>	データを更新する
<code>delete</code>	データを削除する

また、これらのデータは `HashMap` を利用してメモリ上に管理されています。本節では、実際にデータベースを操作しながら、期待どおりに動作することを確認します。

2.5.1 データを登録する

まずは、`insert` コマンドを使ってデータを登録します。

```
Welcome to nekoDB!

db > insert 1 Alice
Inserted.

db > insert 2 Bob
Inserted.

db > insert 3 Carol
Inserted.
```

‘Inserted.’ と表示されれば、データは正常に `HashMap` へ登録されています。

第2章 最小のデータベース

2.5.2 データを取得する

登録したデータを確認するために、‘select’ コマンドを実行します。引数を指定しない場合は、すべてのデータが表示されます。

```
db > select  
  
(1,Alice)  
(2,Bob)  
(3,Carol)
```

特定のデータだけを取得することもできます。

```
db > select 2  
  
Bob
```

キーを指定すると、そのキーに対応するデータだけが表示されます。存在しないキーを指定した場合は、

```
db > select 10  
  
Record not found.
```

となります。

2.5.3 データを更新する

続いて、登録済みのデータを更新します。

```
db > update 2 Robert  
  
Updated.
```

再び検索すると、

2.5 1.5 最小 DB の動作確認

```
db > select 2  
Robert
```

となり、値が更新されていることが確認できます。存在しないキーを更新しようとすると、

```
db > update 10 David  
Record not found.
```

と表示されます。

2.5.4 データを削除する

最後に、不要になったデータを削除してみます。

```
db > delete 1  
Deleted.
```

その後、すべてのデータを表示すると、

```
db > select  
  
(2,Robert)  
(3,Carol)
```

となり、ID が 1 のデータだけが削除されていることが確認できます。削除対象が存在しない場合は、

```
db > delete 5  
Record not found.
```

と表示されます。

第2章 最小のデータベース

2.5.5 エラー入力の確認

これまでの例では、正しいコマンドだけを入力してきました。しかし、実際には誤った入力が行われることもあります。例えば、存在しないコマンドを入力すると、

```
db > hello  
  
Unknown command
```

と表示されます。また、

```
db > insert abc Alice  
  
Error: Key must be an integer.
```

のように、キーへ整数以外を指定した場合も、エラーメッセージを表示して処理を終了します。このように入力内容を検査することで、不正なデータが登録されないようになっています。

2.5.6 現在のデータベースの問題点

ここまでで、CRUD 操作を行えるデータベースが完成しました。しかし、このデータベースには大きな問題があります。例えば、データを登録したあとにプログラムを終了します。

```
db > insert 1 Alice  
  
Inserted.  
  
db > exit
```

再びプログラムを起動すると、新しい 'HashMap' が生成されます。そのため、

```
db > select
```

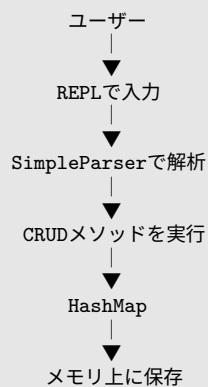
を実行しても、

(何も表示されない)

となり、先ほど登録したデータはすべて失われています。これは、現在のデータベースがデータをメモリ上にしか保持していないためです。プログラムが終了すると、メモリ上のデータも同時に破棄されてしまいます。実際のデータベースでは、サーバーを停止したりプログラムを終了したりしても、登録したデータは保持されなければなりません。そのためには、メモリだけでなく、ディスク上のファイルへデータを保存する仕組みが必要になります。

2.5.7 この章のまとめ

本章では、Java の 'HashMap' を利用して、データベースの最小構成を実装しました。ユーザーからの入力を REPL で受け付け、コマンドを解析し、CRUD 操作を実行することで、「データを登録・取得・更新・削除する」というデータベースの基本機能を実現しました。ここまでの処理の流れをまとめると、次のようになります。



この構成はシンプルで理解しやすく、小規模なデータベースとして十分に機能します。しかし、データがメモリ上にしか存在しないため、プログラムを終了するとすべて失われてしまいます。次章では、この問題を解決するために、データをファイルへ保存する「永続化」の仕組みを実装します。これにより、プログラムを再起動してもデータを保持できる、本格的なデータベースへと一歩近づいていきます。

第 3 章

データの永続化

3.1 2.1 なぜ永続化が必要なのか

第 1 章では、Java の ‘HashMap’ を利用して、データを登録・取得・更新・削除できる最小限のデータベースを実装しました。このデータベースでは、ユーザーが入力したデータはすべて ‘HashMap’ に保存されます。

例えば、

```
db > insert 1 Alice
Inserted.

db > insert 2 Bob
Inserted.
```

と入力すると、データベースの内部では次のような状態になります。

```
HashMap

1 → Alice
2 → Bob
```

この状態であれば、

```
db > select

(1,Alice)
(2,Bob)
```

3.1 2.1 なぜ永続化が必要なのか

と表示され、登録したデータを正しく取得できます。一見すると、これで十分なデータベースが完成したように見えます。しかし、この実装には大きな問題があります。

3.1.1 メモリ上のデータは消えてしまう

現在のデータベースは、すべてのデータをメモリ上で管理しています。そのため、プログラムを終了すると‘HashMap’も同時に破棄されます。例えば、

```
db > insert 1 Alice
Inserted.

db > exit
```

としてプログラムを終了します。その後、再びデータベースを起動して、

```
db > select
```

を実行すると、

(何も表示されない)

となります。登録したはずのデータがすべて消えてしまいました。これは、データがメモリにしか保存されていなかったためです。プログラムが終了すると、メモリ上の情報は失われてしまいます。

3.1.2 実際のデータベースではどうしているのか

実際のデータベースでは、このようなことは起こりません。例えば、住所録や顧客管理システムでデータを登録したあと、パソコンの電源を切ったからといって、データが消えてしまっては困ります。銀行システムであれば、

```
口座番号：12345
残高：500,000円
```

という情報は、サーバーを再起動しても保持されていなければなりません。また、ECサイトで登録された商品情報や注文履歴も、システムを停止するたびに失われることはあ

第3章 データの永続化

りません。このように、プログラムを終了したあともデータを保持し続ける性質を**永続化 (Persistence)**といいます。データベースにとって、永続化は最も重要な機能の一つです。

3.1.3 ファイルへ保存する

永続化を実現する最も単純な方法は、データをファイルへ保存することです。例えば、第1章で‘HashMap’に保存していたデータを、

```
1,Alice  
2,Bob  
3,Carol
```

のようなテキストファイルとして保存します。すると、プログラムを終了してもファイルはディスク上に残ります。次回起動時には、このファイルを読み込んで‘HashMap’を復元すれば、第1章と同じ状態からデータベースを開始できます。処理の流れは次のようになります。

```
insert  
↓  
HashMapへ登録  
↓  
ファイルへ保存
```

そして、起動時には

```
ファイル  
↓  
読み込み  
↓  
HashMapを復元
```

という流れになります。これにより、プログラムを終了してもデータを保持できるようになります。

3.1.4 第2章で実装すること

本章では、第1章で作成したデータベースへ永続化の仕組みを追加します。具体的には、次の機能を実装します。

- データをテキストファイルへ保存する
- 起動時にファイルからデータを読み込む
- データを更新したらファイルにも反映する

これにより、データベースを再起動しても登録したデータを利用できるようになります。ただし、この方法にも課題があります。現在の実装では、データを1件追加するたびにファイル全体を書き直しています。そのため、データ量が増えると更新処理に時間がかかるようになります。まずはシンプルな方法で永続化を実現し、その問題点を確認したうえで、後の章でより効率的な保存方法へ改善していきます。

3.2 2.2 ファイルへの保存

前節では、データを永続化する必要性について説明しました。では、実際にはどこへデータを保存すればよいのでしょうか。本書では、最も単純な方法として、データをテキストファイルへ保存します。プログラムを終了してもファイルはディスク上に残るため、次回起動時に読み込むことでデータベースを復元できます。

3.2.1 保存先を決める

まずは、データベースを保存するファイルを決めます。本書では、プロジェクト内の‘data’ディレクトリにデータベースファイルを配置します。

```
project
├── src
└── data
    └── chapter02.db
```

プログラムでは、保存先を定数として定義しています。

第3章 データの永続化

```
private static final String DATA_PATH = "data/chapter02.db";
```

保存場所を定数としてまとめておくことで、保存先を変更したい場合も、この値を書き換えるだけで済みます。

3.2.2 Path によるファイル管理

Java では、ファイルを扱う方法がいくつかあります。本書では、Java NIO が提供する ‘Path’ クラスを利用します。

```
private Path dataPath;
```

コンストラクタでは、

```
dataPath = Path.of(DATA_PATH);
```

を実行し、文字列として定義したパスから ‘Path’ オブジェクトを生成しています。これにより、

```
data/chapter02.db
```

というファイルを、プラットフォームに依存しない方法で扱えるようになります。以降の章では、ファイルの読み込みや書き込みも、この ‘Path’ オブジェクトを利用して行います。

3.2.3 保存先ディレクトリの作成

ファイルへ保存するためには、保存先のディレクトリが存在している必要があります。しかし、初めてプログラムを実行したときは、‘data’ ディレクトリが存在しない場合があります。そこで、コンストラクタでは保存先ディレクトリの有無を確認しています。

```
if (!Files.exists(dataPath.getParent())) {  
    Files.createDirectories(dataPath.getParent());  
}
```

3.2.2 ファイルへの保存

‘dataPath.getParent()’ は、

```
data
```

の部分を取得します。そのディレクトリが存在しなければ、

```
Files.createDirectories(...)
```

によって自動的に作成します。これにより、ユーザーが事前に ‘data’ ディレクトリを作成していなくても、プログラムが必要なディレクトリを準備できるようになります。

3.2.4 ファイルが存在する場合

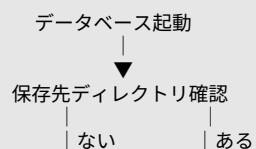
一方で、保存先ディレクトリがすでに存在している場合は、以前に保存されたデータがある可能性があります。そのため、コンストラクタでは次のように処理を分岐しています。

```
if (!Files.exists(dataPath.getParent())) {  
    Files.createDirectories(dataPath.getParent());  
} else {  
    loadFromFile();  
}
```

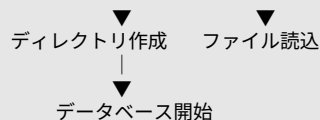
ディレクトリが存在する場合は ‘loadFromFile()’ を呼び出し、保存されているデータを読み込みます。読み込み処理の詳細については、次節で説明します。

3.2.5 ファイル保存の流れ

ここまでで、データを保存するための準備が整いました。データベースは次のような流れでファイルを利用します。



第3章 データの永続化



このように、起動時に保存先を確認することで、初回起動と2回目以降の起動を同じプログラムで扱えるようになります。

3.2.6 この節のまとめ

本節では、データベースを保存するためのファイルとディレクトリを準備しました。

- 保存先を 'data/chapter02.db' に決定した
- 'Path' を利用してファイルを管理した
- 保存先ディレクトリが存在しない場合は自動的に作成した
- 保存先が存在する場合は、以前のデータを読み込めるようにした

これで、データを永続化するための準備が整いました。次節では、'loadFromFile()' を実装し、保存されているデータをメモリ上の 'HashMap' に復元する処理を実装します。

3.3 データ形式の設計

前節では、データベースを保存するためのファイルを用意しました。しかし、ファイルを用意しただけではデータを保存できません。データを保存するためには、どのような形式でデータを書き込むかを決める必要があります。この保存形式をデータ形式と呼びます。

3.3.1 保存形式を考える

現在のデータベースでは、レコードは 'HashMap<Integer, String>' に保存されています。例えば、次のようなデータが登録されているとします。

ID	Name
1	Alice
2	Bob
3	Carol

メモリ上では、

3.3.2.3 データ形式の設計

```
1 → Alice  
2 → Bob  
3 → Carol
```

という状態になっています。この内容をそのままファイルへ保存することはできません。そこで、人間にも読みやすいテキスト形式へ変換して保存します。

3.3.2 key,value 形式で保存する

本書では、1 レコードを

```
キー,値
```

という形式で保存します。例えば、

```
1,Alice
```

は、

```
キー = 1  
値   = Alice
```

という意味になります。複数のレコードがある場合は、1 レコードを 1 行として保存します。

```
1,Alice  
2,Bob  
3,Carol
```

これが、本章で利用するデータベースファイルの内容です。CSV（Comma-Separated Values）のように、項目をカンマで区切り、1 行を 1 レコードとして表しています。

3.3.3 保存時の変換

データを保存するときは、‘HashMap’ の内容を文字列へ変換します。例えば、

第3章 データの永続化

```
1 → Alice
```

というデータは、

```
1,Alice
```

という1行へ変換されます。コードでは、`entrySet()` を利用してすべてのレコードを取り出しています。

```
List<String> lines =  
    db.entrySet().stream()  
        .map(entry -> entry.getKey() + "," + entry.getValue())  
        .toList();
```

処理の流れは次のようになります。

```
HashMap  
  
1 → Alice  
2 → Bob  
3 → Carol  
    ↓  
文字列へ変換  
    ↓  
1,Alice  
2,Bob  
3,Carol
```

最後に、この文字列の一覧をファイルへ書き込みます。

3.3.4 読み込み時の変換

起動時には、逆の処理を行います。まず、ファイルを1行ずつ読み込みます。

```
1,Alice  
2,Bob  
3,Carol
```

続いて、各行をカンマで分割します。

3.3 2.3 データ形式の設計

```
String[] parts = line.split(",");
```

例えば、

```
1,Alice
```

は、

```
parts[0] = "1"  
parts[1] = "Alice"
```

という 2 つの文字列になります。キーは文字列のままでは利用できないため、

```
Integer.parseInt(parts[0])
```

によって整数へ変換します。その後、

```
db.put(Integer.parseInt(parts[0]), parts[1]);
```

を実行することで、‘HashMap’ を復元できます。処理の流れは次のようになります。

```
1,Alice  
2,Bob  
3,Carol  
  ↓  
split(",")  
  ↓  
("1","Alice")  
  ↓  
HashMap  
  
1 → Alice  
2 → Bob  
3 → Carol
```

第3章 データの永続化

保存時とは逆の変換を行うことで、前回終了時と同じ状態のデータベースを復元できます。

3.3.5 不正なデータへの対応

ファイルは人が直接編集することもできます。例えば、

```
1,Alice
2
3,Carol
```

のように、項目が不足した行が含まれているかもしれません。そのまま読み込むとエラーが発生してしまうため、`loadFromFile()` では例外処理を行っています。

```
try {
    String[] parts = line.split(",");
    db.put(Integer.parseInt(parts[0]), parts[1]);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Skipped invalid record format: " + line);
}
```

正しい形式でないレコードは読み飛ばし、

```
Skipped invalid record format: 2
```

というメッセージを表示します。このように入力データを検査することで、一部のレコードが壊れていても、残りのデータは利用できるようになります。

3.3.6 この節のまとめ

本節では、データベースをファイルへ保存するためのデータ形式を設計しました。本章では、次のようなシンプルなテキスト形式を採用します。

```
1,Alice
2,Bob
3,Carol
```

保存時には‘HashMap’をこの形式へ変換し、読み込み時には逆の変換を行うことで、データベースの内容を復元します。このように保存形式をシンプルにすることで、実装を理解しやすくし、ファイルの内容をテキストエディタで直接確認できるという利点もあります。次節では、このデータ形式を利用して、データベースの更新内容をファイルへ書き出す‘saveToFile()’メソッドを実装していきます。

3.4 2.4 更新処理の実装

前節では、データベースファイルの保存形式として、‘キー, 値’のテキスト形式を採用しました。しかし、保存形式を決めただけでは、データベースの内容はファイルへ反映されません。データベースへレコードを追加・更新・削除したときに、その内容をファイルへ書き出す処理が必要になります。本節では、その役割を担う‘saveToFile()’メソッドを実装します。

3.4.1 保存処理の流れ

現在のデータは、すべて‘HashMap’に保存されています。例えば、

```
1 → Alice
2 → Bob
3 → Carol
```

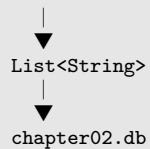
という状態になっているとします。これをそのままファイルへ保存することはできません。そこで、まず各レコードを文字列へ変換し、

```
1,Alice
2,Bob
3,Carol
```

という一覧を作成します。最後に、その一覧をファイルへ書き込みます。処理の流れを図で表すと、次のようになります。

```
HashMap
  ↓
レコードを文字列へ変換
```

第3章 データの永続化



```
graph TD; A[ ] --> B[List<String>]; B --> C[chapter02.db];
```

3.4.2 レコードを文字列へ変換する

保存処理では、‘HashMap’ の各要素を

キー, 値

という文字列へ変換します。コードでは、‘entrySet()’ を利用して Map のすべての要素を取得しています。

```
List<String> lines =
    db.entrySet()
      .stream()
      .map(entry -> entry.getKey() + "," + entry.getValue())
      .toList();
```

ここで、

1 → Alice

というデータは、

1,Alice

という 1 行へ変換されます。同様に、

1 → Alice
2 → Bob
3 → Carol

という Map は、

```
1,Alice  
2,Bob  
3,Carol
```

という文字列の一覧になります。この一覧が、そのままデータベースファイルの内容になります。

3.4.3 ファイルへ書き込む

文字列へ変換したあとは、`Files.write()` を利用してファイルへ保存します。

```
Files.write(dataPath, lines);
```

‘dataPath’ は第 2.2 節で作成した保存先を表す ‘Path’ オブジェクトです。
‘Files.write()’ は、指定したファイルへ文字列の一覧を書き込みます。実行後の
‘chapter02.db’ は次のようになります。

```
1,Alice  
2,Bob  
3,Carol
```

これで、データベースの内容がディスクへ保存されました。

3.4.4 CRUD 操作と保存処理

データベースの内容が変化するのは、

- insert
- update
- delete

の 3 つの操作です。そのため、それぞれのメソッドの最後で `saveToFile()` を呼び出しています。例えば、`insert()` では、

第3章 データの永続化

```
db.putIfAbsent(id, value);  
  
saveToFile();  
  
System.out.println("Inserted and saved to disk.");
```

という流れになっています。同様に、‘update()’では、

```
db.put(id, value);  
  
saveToFile();  
  
System.out.println("Updated and saved to disk.");
```

‘delete()’では、

```
db.remove(id);  
  
saveToFile();  
  
System.out.println("Deleted and saved to disk.");
```

となっています。一方、‘select’はデータを参照するだけで内容を変更しないため、保存処理は不要です。そのため、

コマンド	ファイル保存
insert	○
select	×
update	○
delete	○

という構成になっています。

3.4.5 更新のたびに保存する理由

本書では、データを変更するたびにファイルへ保存しています。例えば、

```
db > insert 1 Alice
```


を実行すると、

```
HashMapへ登録
  ↓
saveToFile()
  ↓
chapter02.db を更新
```

という流れになります。このようにすることで、プログラムが途中で終了しても、直前までのデータがディスクへ保存されています。実装も非常にシンプルで、処理の流れを理解しやすいという利点があります。

3.4.6 この実装の課題

この方法は分かりやすい反面、大きな欠点があります。例えば、100 万件のレコードが登録されているとします。この状態で、

```
db > insert 1000001 Dave
```

を実行すると、追加されたのは 1 件だけですが、

```
1,Alice
2,Bob
3,Carol
...
1000001,Dave
```

という**ファイル全体**を書き直しています。つまり、

```
変更したレコード：1件
書き込むレコード：100万1件
```

となります。データ量が少ない間は問題ありませんが、レコード数が増えるにつれて保存時間も長くなってしまいます。これは現在の実装が「**変更部分だけを書き換える**」のではなく、「**毎回ファイル全体を書き換える**」方式を採用しているためです。

第3章 データの永続化

3.4.7 この節のまとめ

本節では、`saveToFile()` を実装し、`HashMap` の内容をファイルへ保存する仕組みを作成しました。更新処理の流れは次のようになります。



これで、データベースを更新するたびに、その内容がディスクへ保存されるようになりました。ただし、この実装では**更新のたびにファイル全体を書き直している**ため、データ量が増えると保存効率が悪くなります。この課題が、第3章でページ単位 of データ管理を導入する動機となります。

3.5 2.5 永続化の確認

前節までで、データベースへ永続化の仕組みを追加しました。これにより、データは `HashMap` に保存されるだけでなく、ディスク上のファイルにも保存されるようになりました。本節では、実際にデータベースを操作し、プログラムを再起動したあともデータが保持されていることを確認します。

3.5.1 データを登録する

まずはデータベースを起動し、いくつかのレコードを登録します。

```
Welcome to nekoDB!

db > insert 1 Alice
Inserted and saved to disk.
```

3.5 2.5 永続化の確認

```
db > insert 2 Bob
Inserted and saved to disk.

db > insert 3 Carol
Inserted and saved to disk.
```

第 1 章では ‘Inserted.’ と表示していましたが、第 2 章では登録と同時にファイルへの保存も行われるため、

```
Inserted and saved to disk.
```

というメッセージが表示されます。この時点で、メモリ上の ‘HashMap’ だけでなく、データベースファイルにも同じ内容が保存されています。

3.5.2 データベースファイルを確認する

保存されたファイルをテキストエディタで開くと、次のような内容になっています。

```
1,Alice
2,Bob
3,Carol
```

1 行が 1 レコードとなっており、

```
キー,値
```

という形式で保存されていることが確認できます。プログラム内部では ‘HashMap’ として管理されていますが、ファイル上ではシンプルなテキスト形式へ変換されていることが分かります。

3.5.3 プログラムを再起動する

次に、データベースを終了します。

```
db > exit
Bye!
```

第3章 データの永続化

ここでプログラムは終了しますが、ファイルはディスク上に残っています。その後、再びデータベースを起動します。起動時にはコンストラクタから `loadFromFile()` が呼び出され、保存されていたデータが `HashMap` に読み込まれます。

その状態で、

```
db > select
```

を実行すると、

```
(1,Alice)
(2,Bob)
(3,Carol)
```

と表示されます。これは、第1章とは異なり、前回終了時のデータが正しく復元されたことを意味しています。

3.5.4 更新・削除も保存される

更新や削除についても同様です。例えば、

```
db > update 2 Robert
Updated and saved to disk.
```

を実行すると、ファイルの内容は

```
1,Alice
2,Robert
3,Carol
```

へ更新されます。続いて、

```
db > delete 1
Deleted and saved to disk.
```

を実行すると、ファイルの内容は

3.5 2.5 永続化の確認

```
2,Robert  
3,Carol
```

となります。再びデータベースを起動しても、

```
db > select
```

の結果は

```
(2,Robert)  
(3,Carol)
```

となり、更新内容や削除内容も保持されていることが確認できます。

3.5.5 第1章との違い

第1章と第2章のデータベースを比較すると、最も大きな違いはデータの保存場所です。

第1章	第2章
HashMapのみ	HashMap + ファイル
終了すると消える	終了しても残る
起動時は空	起動時にファイルから復元

データベース内部では、引き続き‘HashMap’を利用しています。しかし、第2章では更新のたびにファイルへ保存し、起動時に読み込むことで、永続化を実現しました。このように、「メモリ上で高速に処理しながら、必要に応じてディスクへ保存する」という考え方は、多くのデータベースでも採用されています。

3.5.6 現在の実装の課題

永続化によって、プログラムを終了してもデータは失われなくなりました。しかし、新たな問題もあります。現在の‘saveToFile()’は、データを1件追加しただけでも、データベース全体をファイルへ書き直しています。例えば、

第3章 データの永続化

```
db > insert 1000001 Dave
```

を実行すると、追加されるレコードは1件だけですが、ファイル全体を書き換えることになります。データ件数が数件であれば問題ありませんが、数十万件、数百万件と増えていくと、更新のたびに大量のデータを書き込むことになり、処理時間が大きく増加してしまいます。実際のデータベースでは、このような非効率な方法は採用されていません。変更があった部分だけを書き換えられるように、データを一定サイズの単位で管理しています。

3.5.7 この章のまとめ

本章では、データベースへ永続化の仕組みを追加しました。更新時には‘saveToFile()’によってデータをファイルへ保存し、起動時には‘loadFromFile()’によってファイルからデータを復元することで、プログラムを終了してもデータを保持できるようになりました。処理の流れをまとめると、次のようになります。



これで、本書のデータベースは「データを保持できるデータベース」へと進化しました。しかし、更新のたびにファイル全体を書き換えるという課題が残っています。次章では、この問題を解決するために、データをページ単位で管理する仕組みを導入し、必要な部分だけを読み書きできるデータベースへ改善していきます。

3.6 2.6 第 2 章のまとめと課題

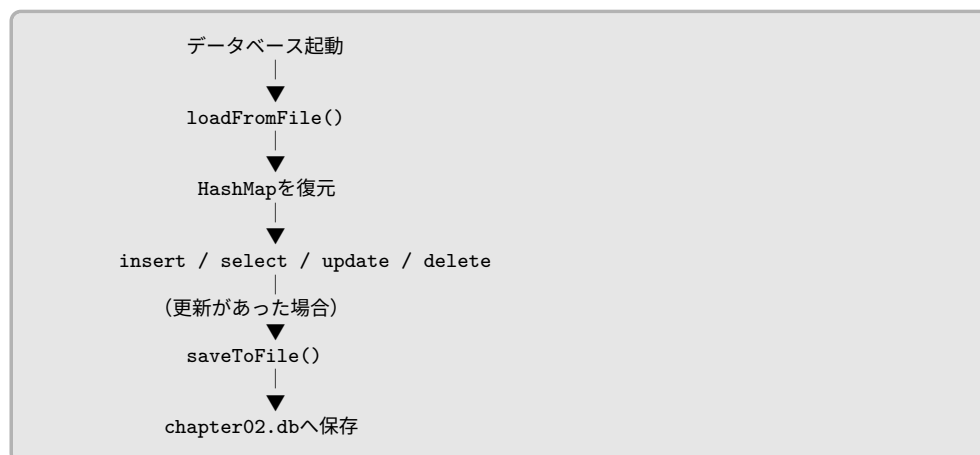
本章では、第 1 章で作成したメモリ上のデータベースに、**永続化**の仕組みを追加しました。第 1 章では、データは ‘HashMap’ のみに保存されていたため、プログラムを終了するとすべて失われていました。第 2 章では、この問題を解決するために、データベースファイルを利用してデータを保存・復元する仕組みを実装しました。

3.6.1 第 2 章で実装した内容

本章では、次の機能を追加しました。

- データベースファイルの保存先を決定する
- 起動時にファイルからデータを読み込む
- 更新時にファイルへ保存する
- ‘HashMap’ とファイルを相互に変換する

処理の流れは次のようになります。



この構成によって、データベースを終了しても、登録したデータを保持できるようになりました。

3.6.2 第 1 章との比較

ここで、第 1 章と第 2 章の違いを整理してみましょう。

第3章 データの永続化

項目	第1章	第2章	
データ管理	HashMap	HashMap	
保存場所	メモリのみ	メモリ+ファイル	
再起動後のデータ	失われる	復元される	
永続化	×	○	

内部でデータを管理しているのは引き続き ‘HashMap’ ですが、ディスク上のファイルと同期することで、永続化を実現しました。このように、「メモリ上で処理を行い、必要に応じてディスクへ保存する」という構成は、多くのデータベースでも基本となる考え方です。

3.6.3 新たに見えてきた課題

一方で、第2章の実装には新たな問題があります。現在の ‘saveToFile()’ では、データが1件だけ変更された場合でも、データベースファイル全体を書き直しています。例えば、100万件のレコードが保存されている状態で、

```
db > update 100 Robert
```

を実行したとしても、変更されるのは1件だけです。しかし、実際には100万件すべてを文字列へ変換し、ファイル全体を書き換えています。処理の流れは次のようになります。



データ量が少ないうちは問題ありませんが、レコード数が増えるにつれて、更新処理にかかる時間も長くなってしまいます。実際のデータベースでは、このような方法は採用されません。

3.6.4 なぜ非効率なのか

ファイルは基本的に先頭から順番に読み書きを行います。そのため、テキスト形式でデータを保存すると、途中の 1 件だけを書き換えることが難しくなります。例えば、

```
1,Alice  
2,Bob  
3,Carol
```

というファイルがあったとします。ここで、

```
2,Robert
```

へ変更したい場合でも、文字数が変わることによって後続のデータの位置がずれてしまいます。その結果、ファイル全体を書き直す必要が生じます。この問題を解決するには、レコードの位置が変化しないような保存方法が必要になります。

3.6.5 次章へ

第 3 章では、この課題を解決するために、データをページ (Page) という固定サイズの単位で管理する仕組みを導入します。ページ単位で管理することで、

- 必要な部分だけを読み込める
- 必要な部分だけを書き換えられる
- データ量が増えても効率よく更新できる

という利点が得られます。これは後の章でインデックスや B+ 木を実装する際の基盤にもなる重要な仕組みです。第 2 章では「データを失わないデータベース」を実現しました。そして第 3 章では、「大量のデータでも効率よく扱えるデータベース」へと発展させていきます。

第4章

ページ (Page) によるデータ配置の改善

4.1 第2章までの課題：ファイル全体の書き直し

第2章では、プログラムを終了してもデータが消えないように、レコードをファイルへ保存する仕組みを作りました。データはメモリ上の `Map<Integer, String>` で管理し、起動時にファイルを読み込んで復元、`insert/update/delete` によりレコードを1件でも変更したら `saveToFile` ですべてのレコードをテキスト形式でファイルへ書き直す、という実装です。

▼ 第2章の保存処理：全レコードをファイルに書き直す（一部省略）

```
private void saveToFile() {
    List<String> lines =
        db.entrySet().stream()
            .map(entry -> entry.getKey() + "," + entry.getValue())
            .toList();
    Files.write(dataPath, lines);
}
```

この実装では1件のレコードを変更するだけでも、ファイル全体を書き直しているという点に無駄があります。10000件のレコードがある状態で1件を更新すると、変更のなかった9999件も含めてすべてを書き出すことになります。書き込む量はレコードの総数に比例して増えていくため、データが増えるほど1回の更新が重くなっていきます。

この無駄が生まれる根本の原因は、**保存の単位がファイル全体になっている**ことにあります。データベースが扱うデータは、ファイルの先頭から末尾まで1本のテキストとして並んでいるだけなので、途中の1件だけを差し替える手段がありません。

ブロックという単位でまとめて入出力を行っています。1バイトだけを書き換えたつも

りでも、そのバイトを含むブロック全体が読み書きされます。したがって、データベースの側もこのブロックに合わせた単位でデータを配置し、読み書きすれば、ディスクとの入出力を無駄なく行えます。

本章では、この「ブロックに合わせた単位」をページとして導入し、更新のたびにファイル全体を書き直すのではなく、**変更のあった部分だけを読み書きする実装**へと作り替えていきます。

4.2 ページという単位の導入

ページとは、ファイルを一定の大きさで区切った固定長のブロックのことです。本章のプログラムでは、1 ページを 4096 バイト (4KB) とします。この値はディスクや OS が扱うブロックの大きさに合わせたもので、多くの環境で自然な単位になります。

ファイル全体を 1 本のデータとして扱う代わりに、先頭から 4096 バイトずつ「ページ 0」「ページ 1」「ページ 2」……と区切って管理します。こうしておくと、ファイルの先頭からのバイト位置は「ページ番号 × 4096」で計算できるため、RandomAccessFile の seek を使って目的のページへ直接移動し、そのページだけを読み書きできます。

▼ ページに関する定数

```
// ページサイズとスロットの定数
private static final int PAGE_SIZE = 4096; // 4KB
private static final int SLOT_SIZE = 64;
private static final int MAX_SLOTS = PAGE_SIZE / SLOT_SIZE; // 64
```

ファイルは第 2 章のテキスト形式ではなく、バイト列を直接扱う RandomAccessFile として読み書きモードで開きます。ファイルの現在の大きさが分かれば、そこに何ページ分のデータが入っているかは次のように求められます。

▼ ファイルサイズからページ数を求める

```
int numPages = (int) Math.ceil((double) file.length() / PAGE_SIZE);
```

これで、ファイルを「ページの集まり」として捉える準備ができました。次は、1 つのページの中にレコードをどう並べるかを決めます。

4.3 固定長スロットによるレコードの設計

1 ページ (4096 バイト) の中を、さらに 64 バイトずつの区画に区切ります。この 1 区画をスロットと呼び、1 つのスロットに 1 件のレコードを格納します。1 ページは 4096

第4章 ページ (Page) によるデータ配置の改善

/ 64 = 64 個のスロットを持ちます。

```
ページ0 (4096バイト)
+-----+-----+-----+ ... +-----+
| スロット0 | スロット1 | スロット2 | ... | スロット63 |
| 64バイト | 64バイト | 64バイト | ... | 64バイト |
+-----+-----+-----+ ... +-----+
```

スロットを**固定長**にしているのには理由があります。1 件ごとの長さがまちまちだと、 n 番目のレコードがファイルのどこから始まるのかを知るために、先頭から順にレコードの区切りを数えていく必要があります。スロットの大きさを 64 バイトに固定しておけば、 s 番目のスロットの位置は「 $s \times 64$ 」で即座に計算できるため、区切りを探す処理が一切いりません。

その代わり、レコードの実体が 64 バイトに満たなくても 1 スロットを丸ごと消費し、値の長さにも上限が生まれます。ここでは扱いやすさを優先し、この固定長の制約を受け入れます。

スロットの 64 バイトの内訳は次のとおりです。

▼表 4.1 スロット (64 バイト) の内訳

オフセット	サイズ	内容
0	1 バイト	有効フラグ (1 = 使用中、0 = 空き)
1	4 バイト	キー (int、ユーザー ID)
5	4 バイト	値の長さ (int、バイト数)
9	55 バイト	値の本体 (UTF-8、最大 55 バイト)

先頭の 1 バイトは**有効フラグ**です。そのスロットが使用中か空きかを表し、レコードを読み書きするときはまずこのフラグを確認します。続く 4 バイトにキー (整数)、次の 4 バイトに値のバイト数、残りの 55 バイトに値の本体を格納します。1 + 4 + 4 + 55 でちょうど 64 バイトになります。

Java でこのバイト列を組み立てるには `ByteBuffer` を使います。ページを表すバイト配列の中から、対象スロットの位置 (`slot × SLOT_SIZE`) を先頭として `ByteBuffer` を用意し、フラグ・キー・長さ・本体の順に書き込みます。

▼ スロットへ 1 件のレコードを書き込む

```
int offset = targetSlot * SLOT_SIZE;
ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
bb.put((byte) 1); // 有効フラグをセット
```

```
bb.putInt(id); // キーをセット

byte[] valBytes = value.getBytes(StandardCharsets.UTF_8);
int len = Math.min(valBytes.length, 55); // 最大55バイトに制限
bb.putInt(len);
bb.put(valBytes, 0, len);
```

値は `Math.min(valBytes.length, 55)` で最大 55 バイトに切り詰めています。これは、値の本体に割り当てられた領域が 55 バイトだからです。読み出すときは、保存しておいた「値の長さ」の分だけバイトを取り出して文字列に戻します。

4.4 ページ単位での読み書きの実装

スロットの設計が決まったので、CRUD（挿入・検索・更新・削除）をページ単位の読み書きで実装していきます。4つのコマンドはどれも、**ページを読み込む → スロットを調べる（または書き換える） → 必要ならページを書き戻す**という共通の流れで動きます。ここでの要点は、書き込みが必要なときでも、ファイルへ書き戻すのは変更のあった 1 ページだけだという点です。

以降、コマンドごとに処理の流れを順を追って見ていきます。

4.4.1 挿入 (insert)

`insert` は、新しいレコードをファイルに 1 件書き加えるコマンドです。処理は次の手順で進みます。

1. 現在のファイルサイズから、ページ数を求める。
2. すべてのページ・スロットを調べ、同じキーがすでに存在しないか確認し、最初に見つかった空きスロットの位置を覚えておく。
3. 空きスロットが 1 つも無ければ、新しいページを 1 枚増やしてその先頭スロットを使う。
4. 書き込み先のページを読み込み、スロットにレコードを書き込む。
5. そのページだけをファイルへ書き戻す。

1~3 が、書き込み先を決める部分です。ページを 1 枚ずつ読み込みながら、各スロットの先頭バイト（有効フラグ）を見て「使用中」か「空き」かを判断します。使用中スロットならキーを取り出して重複していないか確かめ、空きスロットなら（まだ候補を記録していなければ）その位置を書き込み候補として覚えます。

第4章 ページ (Page) によるデータ配置の改善

▼ 重複チェックと空きスロットの探索

```
int numPages = (int) Math.ceil((double) file.length() / PAGE_SIZE);

int targetPage = -1;
int targetSlot = -1;

for (int p = 0; p < numPages; p++) {
    byte[] buf = new byte[PAGE_SIZE];
    file.seek((long) p * PAGE_SIZE);
    file.read(buf);

    for (int s = 0; s < MAX_SLOTS; s++) {
        int offset = s * SLOT_SIZE;
        if (buf[offset] == 1) { // 使用中スロット
            ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
            bb.get(); // 有効フラグをスキップ
            if (bb.getInt() == id) {
                System.out.println("Key already exists. Use update command to modify.");
                return;
            }
        } else if (buf[offset] == 0 && targetPage == -1) { // 最初の空きスロットを記録
            targetPage = p;
            targetSlot = s;
        }
    }
}

// 空きがない場合は新しいページを用意する
if (targetPage == -1) {
    targetPage = numPages;
    targetSlot = 0;
}
```

`targetPage` と `targetSlot` は「これから書き込むページ番号とスロット番号」です。初期値の `-1` は「まだ書き込み先が決まっていない」という目印で、空きスロットを最初に見つけたときにその位置が入ります。ループを最後まで回しても `targetPage` が `-1` のままなら、既存のページがすべて埋まっているということなので、ファイルの末尾に新しいページ（ページ番号は `numPages`）を足し、その先頭スロット（0 番）を使います。

書き込み先が決まったら、そのページを読み込み、スロットにレコードを書き込みます。1 件を書き込む処理は「固定長スロットによるレコードの設計」で見たとおり、有効フラグ・キー・値の長さ・値の本体の順に `ByteBuffer` で並べるだけです。最後に、変更したページだけをファイルへ書き戻します。

▼ 対象ページを読み込み、書き込み、書き戻す

```
// 対象のページを読み込む（新規ページなら読み込みは不要）
byte[] buf = new byte[PAGE_SIZE];
if (targetPage < numPages) {
```

```

    file.seek((long) targetPage * PAGE_SIZE);
    file.read(buf);
}

// スロットにレコードを書き込む
int offset = targetSlot * SLOT_SIZE;
ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
bb.put((byte) 1); // 有効フラグ
bb.putInt(id); // キー
byte[] valBytes = value.getBytes(StandardCharsets.UTF_8);
int len = Math.min(valBytes.length, 55);
bb.putInt(len); // 値の長さ
bb.put(valBytes, 0, len); // 値の本体

// 対象ページだけをファイルに上書き保存
file.seek((long) targetPage * PAGE_SIZE);
file.write(buf);

```

新しいページを作る場合（`targetPage` が `numPages` と等しいとき）は、まだファイルに存在しないページなので読み込みをスキップし、空のバイト配列にそのまま書き込みます。既存ページの空きスロットに書く場合は、他のスロットのデータを消さないよう、いったんページ全体を読み込んでから対象スロットだけを書き換えます。

第2章では更新のたびに全レコードを書き出していました。ここで最後にファイルへ書き込んでいるのは 4096 バイト、つまり 1 ページ分だけです。データベース全体に何万件のレコードがあっても、1 回の `insert` で書き込む量は常に 1 ページで一定になります。ファイル全体の書き直しという無駄は、これで解消できました。

4.4.2 検索 (select)

まず全件表示から見ていきます。処理は次の手順で進みます。

1. 現在のファイルサイズから、ページ数を求める。
2. ページを先頭から順に 1 枚ずつ読み込む。
3. ページ内の各スロットの有効フラグを見て、1（使用中）のものだけを取り出す（0 の空きスロットは飛ばす）。
4. 取り出したスロットからキーと値を読み出し、表示する。

フラグが 0 のスロットは空きなので飛ばし、使用中のスロットだけを表示します。

▼ 全件を表示する select

```

for (int p = 0; p < numPages; p++) {
    byte[] buf = new byte[PAGE_SIZE];
    file.seek((long) p * PAGE_SIZE);

```

第4章 ページ (Page) によるデータ配置の改善

```
file.read(buf);

for (int s = 0; s < MAX_SLOTS; s++) {
    int offset = s * SLOT_SIZE;
    if (buf[offset] == 1) { // 使用中スロットのみ読み取る
        ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
        bb.get(); // フラグスキップ

        int key = bb.getInt();
        int valLen = bb.getInt();
        byte[] valBytes = new byte[valLen];
        bb.get(valBytes);
        String value = new String(valBytes, StandardCharsets.UTF_8);
        System.out.println("(" + key + ", " + value + ")");
    }
}
```

読み出しの順番は、書き込んだときとまったく同じ「フラグ → キー → 値の長さ → 値の本体」です。bb.get() でフラグを1バイト読み飛ばし、bb.getInt() でキー、もう一度 bb.getInt() で値の長さを取得します。ここで取り出した valLen が重要で、**値の本体は固定の55バイトではなく、この長さの分だけ読み出します**。書き込んだときの長さを覚えているからこそ、余分なバイトを含めずに元の値を正しく復元できるわけです。

キーを指定した検索も、ページとスロットを順に調べていく流れは同じです。処理の手順は次のとおりです。

1. ページを先頭から順に1枚ずつ読み込む。
2. 各使用中スロットのキーを取り出し、指定したキーと一致するか調べる。
3. 一致したらそのレコードの値を表示し、処理を打ち切る。
4. 最後まで一致しなければ「見つからなかった」と表示する。

全件表示との違いは、**目的のキーを持つスロットが見つかった時点で、そのレコードを表示して処理を打ち切る点**です。

▼ キーを指定した検索

```
for (int p = 0; p < numPages; p++) {
    byte[] buf = new byte[PAGE_SIZE];
    file.seek((long) p * PAGE_SIZE);
    file.read(buf);

    for (int s = 0; s < MAX_SLOTS; s++) {
        int offset = s * SLOT_SIZE;
        if (buf[offset] == 1) {
            ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
            bb.get(); // フラグスキップ
            int recordKey = bb.getInt();
```



```
        if (recordKey == id) {
            int valLen = bb.getInt();
            byte[] valBytes = new byte[valLen];
            bb.get(valBytes);
            String value = new String(valBytes, StandardCharsets.UTF_8);
            System.out.println(value);
            return;
        }
    }
}
System.out.println("Record not found.");
```

使用中スロットのキーが指定したキーと一致すれば、値を読み出して表示し、すぐに `return` で探索を打ち切ります。最後まで調べても見つからなければ `Record not found` . と表示します。ここで注目したいのは、**目的のレコードが後ろのページにあるほど、それまでのページをすべて読み込んでから見つかる**という点です。この性質が、本章の最後で述べる課題につながります。

4.4.3 更新 (update)

`update` は、既存のレコードの**値だけ**を新しい値に書き換えるコマンドです。キーそのものは変えません。処理の流れは次のとおりです。

1. ページを順に読み込み、指定したキーを持つ使用中スロットを探す。
2. 見つかったら、そのスロットの「値の長さ」と「値の本体」を新しい値で上書きする。
3. そのページだけをファイルへ書き戻す。

▼ レコードの値を上書きする

```
if (recordKey == id) {
    // 対象を見つけたら同じスロットに上書き
    byte[] valBytes = value.getBytes(StandardCharsets.UTF_8);
    int len = Math.min(valBytes.length, 55);

    bb.putInt(len);
    bb.put(valBytes, 0, len);

    // 対象ページだけを保存
    file.seek((long) p * PAGE_SIZE);
    file.write(buf);

    System.out.println("Updated correctly");
    return;
}
```

第4章 ページ (Page) によるデータ配置の改善

ここで少し細かい点を補足します。このコードに入る直前で、キーを確かめるために `b.get()` (フラグ) と `bb.getInt()` (キー) をすでに実行しています。その結果、`ByteBuffer` の読み書き位置はちょうど「値の長さ」の手前まで進んでいます。そのため、続けて `bb.putInt(len)` と `bb.put(valBytes, 0, len)` を呼ぶだけで、フラグとキーはそのまま残したまま、値の部分だけをきれいに上書きできます。レコードは同じスロットにとどまるので位置を動かす必要はなく、書き戻すのはやはりそのレコードを含む1ページだけです。

4.4.4 削除 (delete)

`delete` は、指定したキーのレコードを削除するコマンドです。ここでは、スロットの中身を実際に消したり、後ろのレコードを前に詰めたりはしません。その代わり、**スロット先頭の有効フラグを0にするだけで削除とみなします**。これを**論理削除**と呼びます。

▼ 有効フラグを0にする論理削除

```
if (recordKey == id) {
    // 対象を見つけたら、先頭フラグを0にして論理削除
    buf[offset] = 0;

    file.seek((long) p * PAGE_SIZE);
    file.write(buf);

    System.out.println("Deleted and saved to disk.");
    return;
}
```

フラグを0にすると、そのスロットは以降「空き」として扱われます。`select` はフラグが1のスロットしか表示しないので論理削除したレコードは検索結果から消え、`insert` は空きスロットを再利用するのでこのスロットは次の挿入で上書きされます。キーや値のバイトはフラグを立て直すまでスロットに残りますが、有効フラグが0である限り無効なデータとして無視されるため、問題はありません。1バイトを書き換えるだけで削除が完了し、ここでも書き戻すのは1ページだけです。

4.4.5 処理時間の計測

本章のREPLでは、コマンドを実行するたびにかかった時間を表示するようにしています。

▼ コマンドの実行時間を計測する

```
// --- 計測開始 ---
long startTime = System.nanoTime();

// (ここでコマンドを実行)

// --- 計測終了 ---
long endTime = System.nanoTime();
double timeMs = (endTime - startTime) / 1_000_000.0;
System.out.printf("Executed in %.3f ms\n", timeMs);
```

この表示によって、それぞれの操作にどれくらいの時間がかかっているかを実際に目で確かめられます。次章に進む前に、レコードを増やしながら検索の時間がどう変化するかを観察しておく、この後の課題が実感しやすくなります。

4.5 まとめと次章への課題

本章では、データベースのデータ配置を次のように改善しました。

- ファイルを 4096 バイトの**ページ**という固定長のブロックに区切り、目的のページへ直接アクセスできるようにした。
- 1 ページを 64 バイトの**固定長スロット**に分割し、s 番目のスロットの位置を「s × 64」で即座に計算できるようにした。
- 挿入・更新・削除で書き込むのは、変更のあった **1 ページ分だけ**になり、ファイル全体を書き直す無駄がなくなった。

一方で検索には依然として課題が残っています。第 3 章までの HashMap によるデータ管理ではなくなったため、キーを指定してレコードを探すときファイルの先頭から順に目的のキーが見つかるまですべてのページとスロットを調べています。挿入時の重複チェックも同じように全体を走査しています。このように、先頭から 1 件ずつ順番に調べていく探索を**線形探索 (Linear Search)**と呼びます。線形探索でかかる時間はレコードの件数に比例します。データが数十件であれば一瞬ですが、数万件、数百万件と増えていくと、1 件を探すだけのために毎回大量のページを読み込むことになり、実用的な速度ではなくなってしまいます。本章で追加した実行時間の表示を使えば、件数を増やすほど検索が遅くなっていく様子を確認できるはずです。

次章では、この問題を解決するために **B-Tree (B 木)**を導入します。ファイルの先頭から順に探すのではなく、探しているキーがファイルのどこにあるのかを一足飛びに知る仕組みを組み込み、検索を高速化していきます。

第 5 章

B-Tree による検索の高速化と領域管理

5.1 第 4 章までの課題：線形探索の限界

第 4 章では、データを「ページ」という固定長のブロック単位で管理するようにデータベースを拡張しました。これにより、ディスクとの細かな入出力 (I/O) が減り、効率的にデータを読み書きできるようになりました。

しかし、現在の実装には検索パフォーマンスにおいて大きな課題が残っています。それは、特定のレコードを探す (select) 際や、更新・削除対象を見つける際に、ファイルの先頭から最後まで、すべてのページとスロットを順番に調べているという点です。

このような検索方法を線形探索 (Linear Search) と呼びます。線形探索の処理時間はデータの件数 N に比例する $O(N)$ となります。データが数十件程度であれば処理時間はごくわずかですが、数万件、数百万件と増えていくと、たった 1 件のデータを検索・更新するだけでも毎回膨大なディスク読み込み (フルスキャン) が発生し、実用的な速度ではなくなってしまいます。「なぜ遅いのか」の理由は、この「毎回すべてのデータに目を通しているから」という点にあります。

5.2 検索を高速化するデータ構造の導入

特定のデータへ直接アクセス (ランダムアクセス) するためには、データ本体とは別に「探しているデータが、ファイルのどこにあるのか」をあらかじめ整理したデータ構造が必要です。

これがどのような情報を保持し、なぜ検索が速くなるのか、具体的な例で考えます。

例えば、100 万件のユーザー情報 (ID、名前、メールアドレス、住所など) が保存され

5.3 B-Tree の実装：RecordId の導入

たデータベースがあるとして、データ本体のファイルは巨大であり、特定の ID を探すために先頭から読み込むのは非常に非効率です。

そこで、データ本体とは別に、以下のような「検索キー」と「データの物理的な場所」のペアだけを抽出した、コンパクトな対応表を作成します。

```
[ 検索キー ]      | [ データの物理的な場所 ]
ユーザーID: 10    | ページ2、スロット4
ユーザーID: 15    | ページ8、スロット1
ユーザーID: 8765  | ページ120、スロット3
...
```

この対応表は「キーと場所」しか持たないためファイルサイズが小さく、キー順に並んでいるなどの検索しやすい形に整理されています。ユーザーが「ID が 8765 のデータを検索したい」と要求した場合、データベースは以下のように動きます。

1. 巨大なデータ本体を読む前に、まずはこのコンパクトな**対応表**を探索し、「ID: 8765」を見つけ出します。
2. そこに書かれている物理的な場所（ページ 120、スロット 3）を取得します。
3. データ本体のファイルから、**ページ 120 だけをピンポイントでディスクから読み込み**、該当スロットのデータを取得します。

現実世界で例えるなら、分厚い専門書の中から特定の用語を探すとき、1 ページ目から順番に読むのではなく、巻末の「索引」を見てページ番号を調べ、直接そのページを開くのと同一アプローチです。データベースにおいても、「キーと場所の対応表」を構築することで検索を大幅に高速化できます。

次節からは、この仕組みを Java のプログラムとしてどのように実装していくのかを見ていきます。

5.3 B-Tree の実装：RecordId の導入

検索を高速化する仕組みを踏まえると、実装にはまず「データの物理的な場所（ディスク上の住所）」を表現する仕組みが必要です。

本章のプログラムでは、ファイル上の「どこ」を示す位置情報として、ページ番号とスロット番号を保持する RecordId クラスを新しく定義します。

▼ 位置情報を表す RecordId

第5章 B-Tree による検索の高速化と領域管理

```
/** レコードの物理的な位置情報を保持する識別子 (Record ID) */
static class RecordId {
    int page;
    int slot;

    RecordId(int page, int slot) {
        this.page = page;
        this.slot = slot;
    }
}
```

検索を高速化するデータ構造の役割は、「ユーザー ID (検索キー)」と、この「RecordId (保存場所)」のペアを大量に記憶し、素早く引き出せるようにすることです。では、この膨大なペアのコレクションを、どのような形で管理すればよいでしょうか。

配列やリストを使用すると、途中にデータを追加・削除するたびに全体をシフトさせる必要があり、非効率です。そこで、データベースの検索を高速化する仕組みとして標準的に使われているのが **B-Tree (B 木)** と呼ばれるデータ構造です。(※実際の商用データベースでは、範囲検索に最適化された派生型である「B+Tree」が主流ですが、本章ではその基礎となる B-Tree を実装します。また、データベースの機能としての正式な「インデックス」という概念については第6章で詳しく扱います。)

5.3.1 データベースが B-Tree を選ぶ最大の理由

「検索を速くする木構造」として「二分探索木 (Binary Search Tree)」がありますが、データベースの世界ではほとんど使われず、B-Tree が採用されます。

その理由は、B-Tree が「**1 つのノードに複数のキーを持てる**」という特徴を持っており、第4章で学んだ「**ページ (ブロック)**」との相性が良いためです。

二分探索木は、「1 つのノードに 1 つのキー」を持ち、枝分かれも常に 2 つです。そのため、データが増えるにつれて木が「縦に深く」成長します。この構造をディスク上に保存した場合、木を 1 階層下るたびに「新しいページをディスクから読み込む (ディスク I/O が発生する)」ことになり、パフォーマンスが低下します。

一方、B-Tree には次のような特徴があります。

- **1 つのノードに複数のキーを持てる (マルチウェイ)** : B-Tree は、1 つのノードに数十～数百個のキーを配列として格納します。データベースでは通常、「1 つのノードのサイズ = 1 ページのサイズ (4KB など)」に設計します。これにより、1 回のディスク読み込みで大量のキーを一気にメモリへ読み込むことができます。
- **階層が極めて浅い (横に広い)** : 1 つのノードにキーを複数持てるということは、それらを区切る枝の数も増えるということです。仮に 1 つのノードに 100 個のキー

5.3 B-Treeの実装：RecordIdの導入

(= 101本の枝)があるとしします。高さ1(ルート)で100件、高さ2で1万件、高さ3で100万件のデータを表現できます。つまり、100万件のデータであってもわずか3回のノード走査(=最大3回のディスクI/O)で目的のデータに到達できます。

- **常にバランスを保つ(平衡木)：**データが追加・削除されて特定のノードが溢れそうになると、B-Treeは自動的にノードを2つに分割(Split)したり、隣接するノードと統合(Merge)したりして、木の形を整えます。これにより、どんなキーを検索しても常に一定の速度($O(\log N)$)が保証されます。

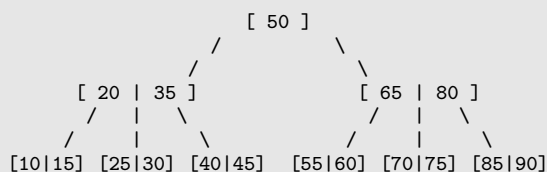
5.3.2 ノードの中身と全体構造のイメージ

B-Treeのノードの中身は、以下のようになっています。複数のキー(Key)が昇順に並んでおり、その「間」や「両端」に子ノードへのポインタ(Pointer)が挟まる構造です。

```
[Pointer 0] Key 1 [Pointer 1] Key 2 [Pointer 2]
```

各ポインタの先には、その範囲に当てはまるキーを持つ子ノードが繋がっています。これを実際のツリー構造として可視化したものが以下の図です。ここでは、高さが3階層あるB-Treeを例にしています。(※見やすさのため、1ノードあたりのキー数を2個までに絞っています)

▼ B-Treeの全体構造(高さ3の例)



一番上にあるのが「ルートノード(根)」、中間にあるのが「内部ノード」、一番下にあるのが「リーフノード(葉)」です。1つのノードに複数のキーが存在することで、進むべきルートが3つ以上に細かく分岐していることが分かります。

この木構造を使って、データを検索する流れを追ってみます。

例1：キー「75」を探す場合 1. まずルートノード(階層1) [50]を確認します。75は「50より大きい」ため、右のポインタへ進みます。2. 次のノード(階層2) [65 | 8

第5章 B-Tree による検索の高速化と領域管理

0] を確認します。75 は「65 より大きく、80 より小さい」ため、真ん中のポインタへ進みます。3. リーフノード（階層 3） [70 | 75] に到達します。ノード内のキーを確認し、「75」を発見したので検索終了（対象の RecordId を取得）となります。

例 2：キー「42」を探す場合（データが存在しないケース） 1. ルートノード（階層 1） [50] を確認します。42 は「50 より小さい」ため、左のポインタへ進みます。2. 次のノード（階層 2） [20 | 35] を確認します。42 は「35 より大きい」ため、右端のポインタへ進みます。3. リーフノード（階層 3） [40 | 45] に到達します。ノード内を確認しますが、42 は存在しません。現在見ているのが末端のリーフノードであるため、これ以上探す場所はないと判断し、検索終了（該当なし）となります。

このように、B-Tree では各ノードにある複数のキーが標識の役割を果たします。ノード内で大小比較を行って進むべき枝を絞り込みながら、目的のデータが存在する末端の葉へと到達するのが、B-Tree による検索メカニズムの基本です。

5.4 B-Tree の実装

前節で学んだ B-Tree の仕組みを、実際の Java コードでどのように表現しているのかを見ていきます。本プログラムでは、BTreeNode（ノード）と、それを管理する BTree（ツリー本体）の 2 つのクラスを使用します。

B-Tree をデータベースのデータ構造として機能させるには、単にノードを定義するだけでなく、検索や更新操作に合わせて木構造のバランスを維持する仕組みが必要です。具体的には以下の 3 つの操作を実装します。

1. **検索 (Search)**: 目的のデータの位置を高速に特定する処理。
2. **挿入と分割 (Split)**: データを追加した際、ノードが満杯になったら 2 つに分割して木を再構成する処理。
3. **削除と補充・統合 (Borrow / Merge)**: データを削除した際、ノード内のデータが不足したら隣接ノードからデータを借りる、または統合してバランスを保つ処理。

以降では、ノードの構造から順に、これら 3 つの操作をどのように実現しているのかを解説します。

5.4.1 ノードの表現 (BTreeNode) と最小次数

まずは、データを格納する「箱」であるノードの定義です。

▼ BTreeNode クラスの実装

```

/** B-tree ノード */
static class BTreeNode {
    int[] keys;           // キー配列 (ユーザーID)
    RecordId[] values;    // レコードポインタ配列 (保存場所)
    BTreeNode[] children; // 子ノードへのポインタ配列
    int numKeys;          // 現在のノードに入っているキーの数
    boolean isLeaf;       // 葉ノード (最下層) かどうか

    BTreeNode(int t, boolean isLeaf) {
        this.isLeaf = isLeaf;
        this.keys = new int[2 * t - 1];           // 最大キー数は 2t - 1
        this.values = new RecordId[2 * t - 1];
        this.children = new BTreeNode[2 * t];      // 最大子ポインタ数は 2t
        this.numKeys = 0;
    }
}

```

コンストラクタの引数 t は「最小次数 (Minimum Degree)」と呼ばれる、B-Tree のサイズを決めるパラメータです。ノードが持てるキーの最大数は $2 * t - 1$ 個となり、子ノードへのポインタの最大数は $2 * t$ 個になります。

例えば $t=10$ の場合、最大で 19 個 ($2 * 10 - 1$) のキーを持てます。

なぜ最大数が「 $2*t - 1$ 」という計算式になっているのでしょうか。これは、ノードが満杯になって「2 つに分割 (Split)」される際の要件を満たすためです。

B-Tree のルールでは、各ノードは最低でも「 $t - 1$ 」個のキーを持たなければならないと決められています。キーの最大数が $2 * t - 1$ であるノードが満杯になり、分割される際の挙動を計算します。

1. 満杯のノードには $2*t - 1$ 個のキーがあります。
2. このノードを「左半分のノード」と「右半分のノード」の 2 つに分割します。
3. 中央にある「1 つのキー」だけは、親ノードに押し上げられます。

数式にすると以下ようになります。

$$\begin{array}{rcl}
 (\text{左半分}) + (\text{右半分}) + (\text{親へ移動}) & = & \text{全体} \\
 (t - 1) + (t - 1) + 1 & = & 2t - 1
 \end{array}$$

$2*t - 1$ を真ん中で割って 1 つを親に渡すと、残された左右のノードのキー数は、どちらも「 $t - 1$ 」になります。これは、B-Tree のルールである「ノードが持つべき最小のキー数」と一致します。つまり、 $2 * t - 1$ というサイズ設定は、分割が発生した直後のノードが最小基準を満たし、木全体のバランス (50% 以上の使用率) を常に一定に保つた

第 5 章 B-Tree による検索の高速化と領域管理

めの設計です。

また、ノード内のキーが k 個あるとき、それを区切る枝の数は必ず $k + 1$ 本になります。そのため、子ノードへのポインタ配列 `children` の最大サイズは、キーの最大数 $2 * t - 1$ に 1 を足した $2 * t$ と定義されています。

5.4.2 検索処理 (Search)

次に、目的のレコードを探す `search` メソッドです。ルート（根）ノードから開始し、再帰的に木を下っていきます。

▼ BTree クラスの検索処理

```
public RecordId search(int key) {
    return root == null ? null : search(root, key);
}

private RecordId search(BTreeNode node, int key) {
    int i = 0;
    // ノード内のキーを順番に比較し、進むべき枝 (インデックス i) を探す
    while (i < node.numKeys && key > node.keys[i]) i++;

    // キーが一致すれば、その位置のRecordIdを返す (発見!)
    if (i < node.numKeys && key == node.keys[i]) {
        return node.values[i];
    }

    // 葉ノードまで到達して見つからなければ、存在しない
    if (node.isLeaf) return null;

    // 対象の子ノードへ進み、再帰的に検索を続ける
    return search(node.children[i], key);
}
```

コード内の `while` ループではノード内の配列 `keys` を先頭から順に調べています。ここで線形探索が行われていますが、ノード内の配列はサイズが小さくすべてメモリ（CPU のキャッシュ）上に連続して存在しているため、ディスクからページを読み込んで探す線形探索とは異なり、短時間で完了します。

配列の中から探しているキーの範囲を見つけ出し、該当する `children[i]` へと進むことで目的のノードを探索します。

5.4.3 挿入とノードの分割 (Split)

B-Tree が常にバランスを保つ仕組みは、データ挿入時のノード分割 (Split) によって実現されています。ノードが保持できるキーの最大数 ($2 * t - 1$) に達して満杯になった場合、そのままでは新しいデータを追加できません。そこで、ノードを 2 つに分割し、真

5.4 B-Tree の実装

ん中のキーを親ノードに押し上げる処理を行います。

最小次数 $t=2$ (キーの最大数 3) の B-Tree を例に、満杯のノード [10 | 20 | 30] に分割が発生する様子を示します。

```
[ 分割前 ]
親ノード:      [ 50 ]
               /
満杯のノード: [ 10 | 20 | 30 ] <-- ここに分割処理(splitChild)を実行

[ 分割後 ]
真ん中の「20」が親へ押し上げられ、残りが左右の子になる
親ノード:      [ 20 | 50 ]
               /   \
新ノード左: [ 10 ]   新ノード右: [ 30 ]
```

実際の Java プログラムにおける splitChild メソッドの実装は以下のようになります。

▼ splitChild メソッドの実装 (一部省略)

```
void splitChild(BTreeNode parent, int i, BTreeNode fullNode) {
    // 1. 右半分を格納する新しいノードを作成
    BTreeNode newNode = new BTreeNode(t, fullNode.isLeaf);
    newNode.numKeys = t - 1;

    // 2. 満杯ノードの「右半分」のキーと値を、新ノードにコピー
    for (int j = 0; j < t - 1; j++) {
        newNode.keys[j] = fullNode.keys[j + t];
        newNode.values[j] = fullNode.values[j + t];
    }
    // (※葉ノードでなければ、子ポインタも同様にコピーします)

    fullNode.numKeys = t - 1; // 満杯ノードは「左半分」だけになる

    // 3. 親ノードのポインタ配列を右に1つズラし、新ノードを挿入する隙間を作る
    for (int j = parent.numKeys; j >= i + 1; j--) {
        parent.children[j + 1] = parent.children[j];
    }
    parent.children[i + 1] = newNode;

    // 4. 親ノードのキー配列も右にズラし、満杯ノードの「真ん中のキー」を押し上げる
    for (int j = parent.numKeys - 1; j >= i; j--) {
        parent.keys[j + 1] = parent.keys[j];
        parent.values[j + 1] = parent.values[j];
    }
    parent.keys[i] = fullNode.keys[t - 1]; // 真ん中のキーを親へ
    parent.values[i] = fullNode.values[t - 1];

    parent.numKeys++; // 親のキー数が1つ増える
}
```

第5章 B-Tree による検索の高速化と領域管理

ここで行われているのは、主に「配列要素の右シフト（隙間づくり）」と「データの移動」です。もしルートノードが満杯になって分割されると、押し上げられたキーを受け取るために新しいルートが作られ、木の高さが1段高くなります。B-Tree は「上から下」へ伸びるのではなく、「下から上」へと持ち上がるように成長していくため、すべての葉ノードまでの深さが常に揃い、バランスが維持されます。

5.4.4 削除処理におけるバランスの維持（Borrow と Merge）

データの削除処理では、挿入時と同様に木構造のバランスを維持するための調整が必要です。B-Tree の各ノード（ルート以外）は、常に最小キー数（ $t - 1$ ）以上を保持していなければならないというルールがあります。データを削除した結果、キーの数がこの最小値を下回ってしまった場合（アンダーフロー）、木を再編成してバランスを取り戻します。そのための手段が「隣接ノードからの補充（Borrow）」と「隣接ノードとの統合（Merge）」です。

隣接ノードからの補充（Borrow）

アンダーフローを起こしたノードの「すぐ隣のノード（隣接ノード）」に余裕がある（キーが t 個以上ある）場合、隣接ノードからキーを1つ移動させます。ただし、直接横に移動させるのではなく、大小関係を保つため、「親のキーを自分に降ろし、隣接ノードのキーを親に引き上げる」という回転（Rotation）動作を行います。

左の隣接ノードからキーを補充する `borrowFromPrev` を例に示します。

```
[ 補充前 ]
親ノード:           [ 40 ]
                  /   \
左の隣接ノード(余裕あり): 対象ノード(不足):
[ 10 | 20 | 30 ]         [ 50 ]

[ 補充後 ]
親の「40」が対象ノードに降り、左隣接ノードの最大値「30」が親に上がる
親ノード:           [ 30 ]
                  /   \
左の隣接ノード:      対象ノード:
[ 10 | 20 ]          [ 40 | 50 ]
```

これを実現するコードは以下ようになります。

▼ `borrowFromPrev` メソッドの実装

```

void borrowFromPrev(BTreeNode node, int idx) {
    BTreeNode child = node.children[idx];    // アンダーフローした対象ノード
    BTreeNode sibling = node.children[idx - 1]; // 左の隣接ノード

    // 1. childのキー配列を右に1つズラして、先頭(0番目)に空きを作る
    for (int i = child.numKeys - 1; i >= 0; --i) {
        child.keys[i + 1] = child.keys[i];
    }

    // 2. 親ノードのキーを、childの先頭に降ろす
    child.keys[0] = node.keys[idx - 1];

    // (※葉ノードでなければ、siblingの最後の子ポインタをchildの先頭に移動します)

    // 3. 左の隣接ノード(sibling)の最後のキーを、親ノードに引き上げる
    node.keys[idx - 1] = sibling.keys[sibling.numKeys - 1];

    child.numKeys++;    // 対象ノードのキーが1つ増えた
    sibling.numKeys--;   // 左の隣接ノードのキーが1つ減った
}

```

(※右の隣接ノードから補充する borrowFromNext も、方向が逆になるだけで基本的なロジックは同じです。)

隣接ノードとの統合 (Merge)

もし、左右どちらの隣接ノードもキーが最小値 ($t - 1$) であり、キーを分ける余裕がない場合は、「対象ノード」と「隣接ノード」、そして「それらを分かつ親ノードのキー」の3つを、1つのノードに統合 (Merge) します。これが分割 (Split) の逆の操作となります。

```

[ 統合前 ]
親ノード:           [ 30 ]
                  /   \
左の対象ノード(不足): [ 10 ]  右の隣接ノード(余裕なし): [ 40 | 50 ]

[ 統合後 ]
親の「30」が降りてきて、すべてが1つのノードにまとまる
親ノード:           [ ] (※キーが1つ減る)
                  /
統合されたノード:   [ 10 | 30 | 40 | 50 ]

```

▼ merge メソッドの実装

```
void merge(BTreeNode node, int idx) {
    BTreeNode child = node.children[idx];          // 左側になるノード
    BTreeNode sibling = node.children[idx + 1];      // 右側の隣接ノード

    // 1. 親のキーを、childの末尾に降ろす（これが両者をつなぐ境界線になる）
    child.keys[t - 1] = node.keys[idx];

    // 2. 右の隣接ノード(sibling)のすべてのキーを、childにコピーする
    for (int i = 0; i < sibling.numKeys; ++i) {
        child.keys[i + t] = sibling.keys[i];
    }

    // （※葉ノードでなければ、子ポインタも同様にコピーします）

    child.numKeys += sibling.numKeys + 1; // キー数が増加

    // 3. 親ノードのキー配列と子ポインタ配列を左に詰めて、空いた隙間を塞ぐ
    for (int i = idx + 1; i < node.numKeys; ++i) {
        node.keys[i - 1] = node.keys[i];
    }
    for (int i = idx + 2; i <= node.numKeys; ++i) {
        node.children[i - 1] = node.children[i];
    }
    node.numKeys--; // 親のキーが1つ減った
}
```

統合を行うと、親ノードのキーが 1 つ減ります。その結果、親ノード自身がアンダーフローを引き起こす可能性があり、その場合は親ノードに対しても再帰的に Borrow や Merge が実行されます。もしルートノードの最後のキーが降ろされて空になった場合は、統合されてできた新しいノードが新たなルートとなり、木全体の高さが 1 段低くなります。

このように、親・隣接ノード間で細かく連携しながら「分割」「補充」「統合」を行うことで、B-Tree はデータの追加・削除を繰り返しても、検索パフォーマンスを維持し続けることができます。

5.5 起動時の B-Tree 構築と空き領域管理

前節までで、B-Tree のバランス調整を含めた実装を確認しました。しかし、現在のプログラムにおいて、B-Tree はメモリ上のみ構築されているという点に注意が必要です。

データベース (nekoDB) を終了すると、メモリ上のデータは揮発してしまいます。そのため、データベースを起動するたびに、ディスク上のデータファイル (chapter05.db) を読み込み、B-Tree を再構築する必要があります。

5.5.1 B-Tree の再構築 (buildTree)

起動時に実行される buildTree メソッドの実装を確認します。ここではファイルの全ページおよび全スロットを走査し、有効なデータが存在すれば B-Tree に登録します。

▼ 起動時の B-Tree 構築処理

```
private void buildTree() throws IOException {
    int numPages = (int) Math.ceil((double) file.length() / PAGE_SIZE);
    int count = 0;

    for (int p = 0; p < numPages; p++) {
        byte[] buf = new byte[PAGE_SIZE];
        file.seek((long) p * PAGE_SIZE);
        file.read(buf);

        for (int s = 0; s < MAX_SLOTS; s++) {
            int offset = s * SLOT_SIZE;

            // スロットの先頭1バイト (フラグ) を確認
            if (buf[offset] == 1) {
                // データが存在する場合はB-Treeに登録
                ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
                bb.get(); // フラグをスキップ
                int recordKey = bb.getInt();

                // B-Treeへキーと位置情報を挿入
                btree.insert(recordKey, new RecordId(p, s));
                count++;
            } else {
                // フラグが0 (空きスロット) の場合はfreeListに追加
                freeList.offer(new RecordId(p, s));
            }
        }
    }
    System.out.println("B-Tree build complete. Loaded " + count + " records.");
}
```

起動時に1度だけファイル全体の線形探索が発生しますが、これによりメモリ上に B-Tree が構築されるため、起動後の検索処理は高速に行われるようになります。

5.5.2 空き領域リスト (freeList) による挿入の高速化

上記の buildTree メソッドの else ブロックでは、データが存在しなかったスロット (未確保、または論理削除されたスロット) の位置情報を、freeList というキュー (Queue) に追加しています。

▼ freeList の定義

第 5 章 B-Tree による検索の高速化と領域管理

```
private Queue<RecordId> freeList;
```

第 4 章までの実装では、新しいデータを挿入 (insert) する際、空きスロットを見つけるためにファイルの先頭から線形探索を行う必要がありました。これは検索処理だけでなく、挿入処理においてもパフォーマンスのボトルネックとなっていました。

この freeList を用いることで、挿入時の探索コストを削減できます。新しいレコードを挿入する際は、キューから要素を 1 つ取得 (freeList.poll()) するだけで、次に書き込むべきディスク上の物理的な位置を特定できます。これにより、ディスクを順次読み込んで空き領域を探す I/O 処理が不要になります。

また、レコードを削除 (delete) した際も、対象の RecordId を freeList に追加 (offer) します。これにより、論理削除によって発生した非連続な空き領域 (フラグメンテーション) を管理し、後続の挿入処理で効率的に再利用することが可能になります。

B-Tree による検索の高速化と、freeList による挿入の高速化によって、データベースの基本操作がどのように改善されたのか、次節で CRUD の実装を確認します。

5.6 B-Tree を用いた CRUD の高速化

前節までに構築した B-Tree と freeList を活用し、実際のデータベース操作 (CRUD: 作成、読み出し、更新、削除) がどのように改善されたのかを確認します。

5.6.1 検索処理 (select) の改善

第 4 章までの select 処理では、対象のキーを見つけるまで全ページを順に読み込む線形探索を行っていました。B-Tree 導入後の select(String key) メソッドは以下のようになります。

▼ B-Tree を利用した select 処理

```
public void select(String key) {
    // (中略: キーの数値変換とエラーハンドリング)

    try {
        // 1. B-Treeから位置情報 (RecordId) を取得
        RecordId rid = btreesearch(id);
        if (rid == null) {
            System.out.println("Record not found.");
            return;
        }

        // 2. 対象のページのみをディスクから読み込む
        byte[] buf = new byte[PAGE_SIZE];
```



```

file.seek((long) rid.page * PAGE_SIZE);
file.read(buf);

// 3. 対象スロットのオフセットを計算し、データを取得
int offset = rid.slot * SLOT_SIZE;
ByteBuffer bb = ByteBuffer.wrap(buf, offset, SLOT_SIZE);
bb.get(); // フラグ
bb.getInt(); // キー
int valLen = bb.getInt();
byte[] valBytes = new byte[valLen];
bb.get(valBytes);

System.out.println(new String(valBytes, StandardCharsets.UTF_8));
} catch (IOException e) {
    System.out.println("Disk I/O Error during select.");
}
}

```

コードからページとスロットを走査する for ループが無くなっていることが分かります。btree.search(id) によって物理的な位置（ページ番号とスロット番号）を特定し、file.seek() を用いて該当ページへ直接アクセスしています。これにより、データ件数が増加しても、検索にかかるディスク I/O は常に 1 ページ分の読み込みのみとなります。

5.6.2 挿入処理 (insert) の改善

データの挿入処理では、B-Tree による重複チェックの高速化と、freeList による空き領域確保の効率化が行われています。

▼ B-Tree と freeList を利用した insert 処理（一部抜粋）

```

// B-treeによる高速な重複チェック O(log N)
if (btree.search(id) != null) {
    System.out.println("Key already exists. Use update command to modify.");
    return;
}

// (中略)

int targetPage = -1;
int targetSlot = -1;

// freeリストから空きスロットを取得
if (!freeList.isEmpty()) {
    RecordId freeId = freeList.poll();
    targetPage = freeId.page;
    targetSlot = freeId.slot;
} else {
    // freeリストが空の場合は新規ページを作成
    targetPage = numPages;
    targetSlot = 0;
    // 新規ページの残りスロットをfreeリストに登録
}

```

第 5 章 B-Tree による検索の高速化と領域管理

```
for (int s = 1; s < MAX_SLOTS; s++) {
    freeList.offer(new RecordId(targetPage, s));
}
}
```

挿入時のキー重複チェックにおいて、以前はファイルを先頭から走査していましたが、B-Tree の検索に置き換えることで処理時間を $O(\log N)$ に短縮しています。また、書き込み先の決定も `freeList` から位置情報を取り出す処理に変更されたため、空き領域を探すためのファイル読み込みが不要になりました。

5.6.3 更新 (update) と削除 (delete) の改善

更新と削除についても同様のアプローチが取られています。両者とも、まずは `btree.search(id)` で対象のレコード位置を特定し、その 1 ページだけを読み込んで処理を行います。

特に削除処理 (delete) では、ディスク上のフラグを 0 (論理削除) にしてファイルに書き戻した後、メモリ上の管理情報を更新する処理が追加されています。

▼ delete 処理における管理情報の更新

```
// B-treeからも削除（ノードのマージ処理等が発生）
btree.delete(id);

// 削除して空いたスロットをfreeListに登録
freeList.offer(rid);
```

データの変更操作が行われた際は、必ずファイル（実データ）とメモリ上の B-Tree・空き領域リスト（管理データ）の両方を同期して更新する必要があります。この同期を正しく行うことで、データベース全体の整合性が保たれます。

5.6.4 実行速度の比較：どれくらい速くなったのか

ここまで実装してきた「B-Tree による検索の高速化」と「`freeList` による空き領域管理」によって、実際のデータベース操作がどれほど劇的に改善されたのかを確認してみましょう。

あらかじめデータベースに 1 万件のデータ (ID: 1~10000) を挿入した状態で、前章（線形探索）と本章（B-Tree + `freeList`）のプログラムに対して同じコマンドを実行し、処理時間を比較しました。

▼表 5.1 実行時間の比較（1 万件のデータが存在する状態）

コマンド	前章（線形探索）	本章（B-Tree + freeList）
select 8765	11.403 ms	0.686 ms
update 8765 test	5.933 ms	2.960 ms
delete 1	1.628 ms	0.991 ms
delete 8765	3.257 ms	0.983 ms
insert 10001 test	3.310 ms	1.218 ms

結果は一目瞭然です。ファイルの先頭から順にページを読み込んでいた `select` 処理は、B-Tree を導入したことで **16 倍以上もの高速化** を果たしています。

特に注目していただきたいのが `delete` の結果です。前章の線形探索では、ファイルの先頭付近にある `delete 1` (1.628 ms) に比べて、末尾付近にある `delete 8765` (3.257 ms) は、対象を見つけるまでに多くのデータを読み込む必要があり、処理時間が 2 倍近くかかっていました。しかし、本章の実装では `delete 1` も `delete 8765` も **どちらも約 0.9 ms** で完了しています。B-Tree は木がバランスを保つ（すべての葉までの深さが同じになる）ため、**データがファイルのどこにあっても、常に一定の速度 ($O(\log N)$) でアクセスできる**という強力な特性が数値として美しく表れています。

また、新しいデータを追加する `insert` についても、挿入前の「キーの重複チェック」が B-Tree で高速化され、さらに書き込み先の確保も「空きスロットを探す線形探索」から「freeList から位置情報を取り出すだけ ($O(1)$)」に変わったことで、処理時間が大幅に短縮されています。

5.7 まとめと次章への課題

本章では、データベースの検索・更新処理を高速化する「B-Tree」と、空き領域を効率的に再利用するための「freeList」を実装しました。

これらを導入したことにより、前章で直面した「線形探索によるパフォーマンスの限界」を見事に克服しました。検索対象のデータを特定する処理が $O(\log N)$ の時間計算量へと改善され、データ挿入時の空き領域探索にかかるディスク I/O も削減されました。これにより、内部的なデータアクセスの仕組みは実践的な速度で動作するようになりました。

しかし、データベースの「内部の処理速度」は向上したものの、ユーザーがデータベースを操作するための「インターフェース」には課題が残されています。

- **操作方法が複雑で扱いにくい:** 現在の仕様では、データの挿入や検索を行うために、専用のメソッドや独自の低レイヤーなコマンドを直接呼び出す必要があります。これではアプリケーションから汎用的かつ直感的に利用することができません。

第 5 章 B-Tree による検索の高速化と領域管理

- 「何を」取得するかは分かるが、「どう」実行するかが決まっていない：現在は「B-Tree の `search` を呼んでからファイルを読む」という単純な手順が決め打ちになっています。今後「特定の条件を満たすレコードだけを取得したい」といった要求が出た場合、ユーザー自身が「どの仕組みを使って、どの順番でデータを読み込むか」という具体的な手続き（手段）まで意識しなければならなくなってしまいます。

現代のリレーショナルデータベースが広く使われている最大の理由は、ユーザーが「欲しいデータ（目的）」だけを宣言的に記述すれば、データベース側が自動的に「最適な検索手順（手段）」を判断して実行してくれる点にあります。

次章（第 5 章）では、この課題を解決するために **SQL** の処理エンジンをデータベースに組み込みます。「何をするか」を記述する標準言語である SQL を読み解くため、字句解析（トークナイズ）や構文解析（パース）、そして抽象構文木（AST）の構築といった、言語処理系の領域へと足を踏み入れます。

まずは第一歩として、**SELECT** 文や **INSERT** 文をプログラムが理解できる形に変換する仕組みを作っていきます。

第 6 章

テーブルが定義され、データが保存されるまで

chapter05 では、ユーザーが入力した SQL が、直接ファイルへ書き込まれるわけではない。入力された SQL は、複数の処理を順番に通過し、最終的にテーブルごとのページファイルへ保存される。

この一連の処理を理解するためには、最初に「テーブルがどのように作られるのか」を知る必要がある。INSERT や SELECT といった SQL は、すでにテーブルが存在していることを前提としているからである。

たとえば、次の INSERT 文を考えてみよう。

▼ INSERT 文の例

```
INSERT INTO users (id, name, age)
VALUES (1, 'Taro', 20);
```

この SQL には、`users` というテーブル名と、`id`、`name`、`age` という列名が含まれている。しかし、データベースが事前に `users` テーブルの定義を知らなければ、それぞれの値をどのように保存すればよいかわからない。

1 は整数として保存するのか、それとも文字列として保存するのか。`Taro` には何バイト分の領域を用意すればよいのか。20 はどの列に対応するのか。これらを決めるために必要なのが、テーブル定義である。

chapter05 では、テーブル定義を `Schema` として表現している。`Schema` は、テーブルが持つ列の名前、データ型、サイズなどを保持する。言い換えれば、`Schema` はテーブルの構造を表す設計情報である。

ただし、`Schema` はプログラムの起動時から最初から存在するわけではない。ユーザー

第 6 章 テーブルが定義され、データが保存されるまで

が CREATE TABLE 文を実行したときに生成される。

次の SQL を例に、テーブルが作られるまでの流れを見ていこう。

▼ users テーブルの定義

```
CREATE TABLE users (  
    id INTEGER,  
    name STRING(20),  
    age INTEGER  
);
```

この SQL は、**users** という名前のテーブルを作り、そのテーブルに三つの列を定義している。**id** と **age** は整数型であり、**name** は最大 20 バイトの文字列型である。

ユーザーがこの SQL を入力すると、最初に **SimpleParser** が呼び出される。ただし、**SimpleParser** がいきなりテーブルを作るわけではない。**Parser** の役割は、入力された SQL を読み取り、その内容を Java プログラムから扱いやすい形へ変換することである。

その処理の流れを図にすると、次のようになる。



Tokenizer によって SQL は、**CREATE**、**TABLE**、**users**、**(**、**id**、**INTEGER** といった小さな要素に分けられる。その後、**SimpleParser** はそれらの並びを読み取り、**CREATE TABLE** 文であることを判断する。

解析が終わると、SQL の内容は `Statement.CreateTable` として表現される。元の SQL 文字列をそのまま保持するのではなく、テーブル名と列定義を整理したデータへ変換するのである。

概念的には、次のような情報が作られる。

```
Statement.CreateTable

テーブル名
  users

列定義
  id      型: INTEGER

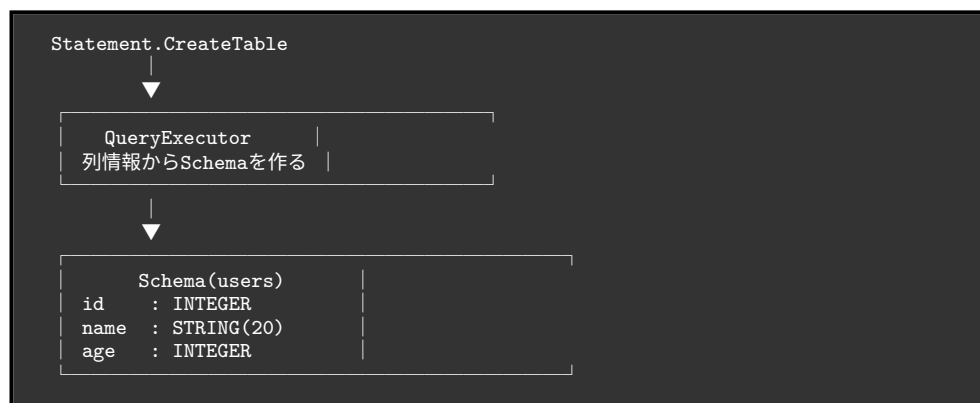
  name    型: STRING
          最大長: 20

  age     型: INTEGER
```

ここまでの処理では、まだテーブルファイルは作成されていない。Parser は SQL の内容を読み取っただけであり、実際のデータベース操作は行っていない。

解析後の `Statement.CreateTable` は、`QueryExecutor` へ渡される。`QueryExecutor` は、受け取った `Statement` の種類を調べ、`CREATE TABLE` 文であればテーブル作成用の処理を実行する。

`QueryExecutor` は、`Statement` に保存されている列情報をもとに `Schema` を生成する。



この `Schema` によって、`users` テーブルがどのような構造を持つかが決まる。`Schema` は、単に列名を保存するだけではない。各列がどの型であるか、ファイル上で何バイト使

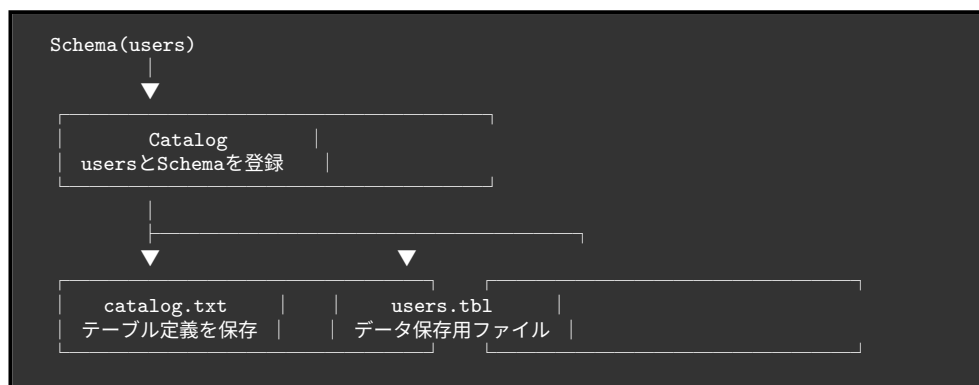
第6章 テーブルが定義され、データが保存されるまで

用するか、レコード全体が何バイトになるかを計算するためにも使われる。

Schema が生成されると、QueryExecutor はその Schema を Catalog へ渡す。

Catalog は、データベース内に存在するテーブルを管理するための仕組みである。どのテーブルが存在し、それぞれがどの Schema を持ち、どのファイルと対応しているかを管理する。

users テーブルを登録するときの流れは、次のようになる。



ここでは、二種類のファイルが登場する。

catalog.txt には、テーブル名や列名、データ型といった定義情報が保存される。一方、users.tbl には、INSERT によって追加される実際のデータが保存される。

この二つは役割が異なる。

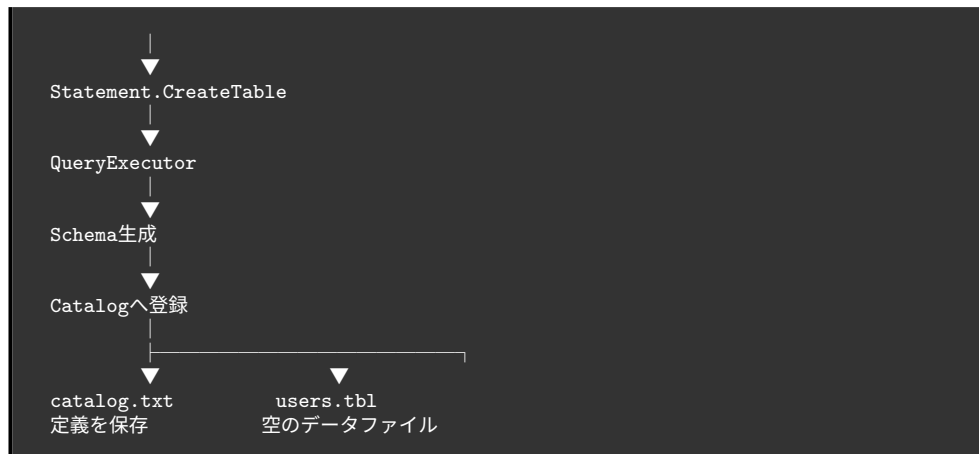
catalog.txt は、データベースの設計情報を保存するファイルである。データベースを再起動したとき、Catalog はこのファイルを読み込み、以前に作成したテーブルの Schema を復元する。

一方、users.tbl は、users テーブルの行データを保存するためのファイルである。CREATE TABLE を実行した直後は、このファイルにはまだレコードが存在しない。データを書き込むための空のファイルだけが用意された状態である。

したがって、CREATE TABLE 文の処理全体は、次のようにまとめられる。



6.1 Schema は後続の処理で繰り返し使われる



`CREATE TABLE` によって作られるのは、データそのものではない。テーブルがどのような列を持ち、どの型で値を保存するのかという設計情報である。

この設計情報が作られて初めて、`INSERT`、`SELECT`、`UPDATE`、`DELETE` といった SQL を実行できるようになる。

6.1 Schema は後続の処理で繰り返し使われる

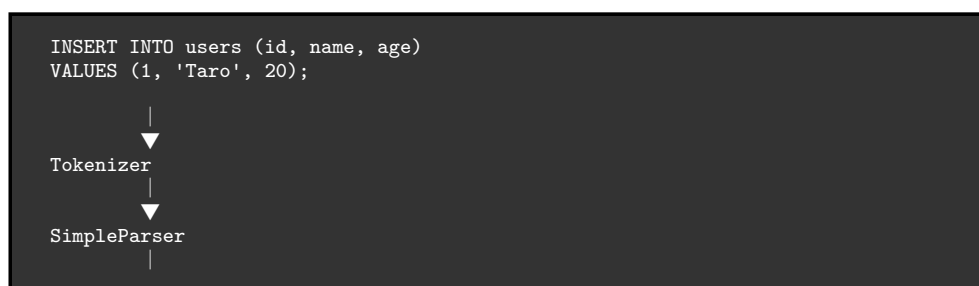
Schema は、`CREATE TABLE` のときだけ使用されるものではない。一度作られた Schema は、その後のデータ操作でも繰り返し参照される。

たとえば、次の `INSERT` 文を実行するとする。

▼ Schema を利用する `INSERT` 文

```
INSERT INTO users (id, name, age)
VALUES (1, 'Taro', 20);
```

この SQL も、最初は `Tokenizer` と `SimpleParser` によって解析される。



第6章 テーブルが定義され、データが保存されるまで

▼
Statement.Insert

解析後の Statement.Insert には、次の情報が保存される。

Statement.Insert

テーブル名
users

列名
id
name
age

値
1
Taro
20

ただし、この段階では、値の型が完全に決まったわけではない。Parser が読み取った 1 や 20 は、SQL 中に書かれていた文字列として扱われている。

そこで QueryExecutor は、Catalog から users テーブルを取得し、その Schema を参照する。

Statement.Insert

▼
QueryExecutor

▼
Catalogからusersを取得

▼
Schemaを参照

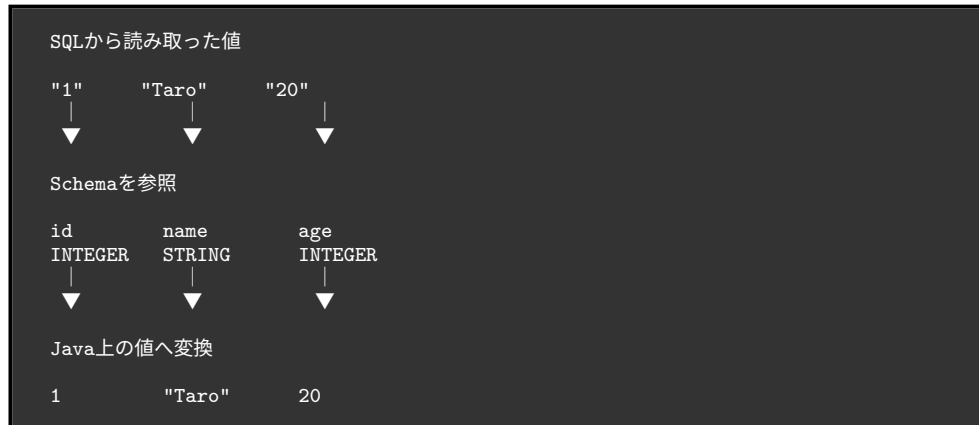
id : INTEGER
name : STRING(20)
age : INTEGER

Schema を見ることで、QueryExecutor は各値をどの型へ変換すればよいか判断できる。

id 列は INTEGER なので、文字列として読み取られた"1"を整数値 1 へ変換する。name 列は STRING なので、"Taro"を文字列として扱う。age 列は INTEGER なので、"20"を整数値 20 へ変換する。

6.1 Schema は後続の処理で繰り返し使われる

この処理を図にすると、次のようになる。



型変換が完了すると、QueryExecutor は一行分のデータを表す Row を生成する。



Row は、Java プログラム上で一行分のデータを扱うための形式である。この時点では、まだファイルへ保存されていない。

QueryExecutor は、生成した Row を `Table.insert()` へ渡す。



Table は、Row をそのままファイルへ書き込むわけではない。Java のオブジェクトを、ファイルへ保存できるバイト列へ変換する必要がある。

第 6 章 テーブルが定義され、データが保存されるまで

たとえば、`users` テーブルの Schema が次のように定義されているとする。

```
id    INTEGER
name  STRING(20)
age    INTEGER
```

このとき、一つのレコードは概念的に次の構造になる。

使用フラグ	id	文字列長	nameの領域	age
1バイト	4バイト	4バイト	20バイト	4バイト

先頭の使用フラグは、そのスロットに有効なレコードが保存されているかを表す。1 なら使用中、0 なら未使用である。

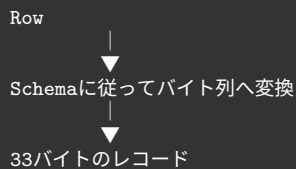
INTEGER 型の `id` と `age` は、それぞれ 4 バイトで保存される。STRING 型の `name` は、実際の文字列長を保存する領域と、最大 20 バイトの文字列領域を使用する。

この場合、レコード全体のサイズは次のようになる。

使用フラグ	1バイト
id	4バイト
nameの文字列長	4バイト
name本体	20バイト
age	4バイト
合計	33バイト

Schema は、このレコードサイズの計算にも使われる。

Table は、Row を Schema の列順に従ってバイト列へ変換し、4KB のページ内へ保存する。



6.1 Schema は後続の処理で繰り返し使われる

↓
4KBページ内の空きスロットへ保存
↓
users.tblへ書き込み

ページとは、データベースがファイルを読み書きする固定サイズの単位である。chapter05 では、1 ページを 4096 バイトとして扱う。

users.tbl

ページ0
4096バイト

slot 0 | slot 1 | slot 2 | ...

ページ1
4096バイト

スロットとは、ページ内に設けられたレコード一件分の固定領域である。

先ほどの例では、一レコードが 33 バイトであるため、一ページに保存できるスロット数は、4096 を 33 で割って求められる。

$$4096 \div 33 = 124 \text{ 残り } 4$$

したがって、一ページには 124 件分のレコードを保存できる。残りの 4 バイトは、一レコード分の領域に満たないため使用されない。

Table は、ページ内のスロットを先頭から調べる。スロットの先頭にある使用フラグが 0 であれば、そのスロットは空いている。空きスロットが見つかったら、Table はそこへ新しいレコードを書き込む。

ページ内のスロット

slot 0

第 6 章 テーブルが定義され、データが保存されるまで

使用フラグ 1	← 使用中
レコード	

slot 1

使用フラグ 0	← 空き
未使用領域	

slot 2

使用フラグ 1	← 使用中
レコード	

この場合、新しい Row は slot 1 へ保存される。

既存のページに空きスロットがなければ、Table はファイルの末尾に新しい 4KB ページを追加する。

以上が、INSERT によって一行のデータが保存されるまでの流れである。

```
INSERT文
  ↓
Tokenizer
  ↓
SimpleParser
  ↓
Statement.Insert
  ↓
QueryExecutor
  ↓
CatalogからTableとSchemaを取得
  ↓
Schemaに従って値を型変換
  ↓
Rowを生成
  ↓
Table.insert()
  ↓
Rowをバイト列へ変換
  ↓
ページ内の空きスロットを検索
```

6.2 SELECT では Schema を使ってレコードを読み戻す

↓
users.tblへ保存

6.2 SELECT では Schema を使ってレコードを読み戻す

次に、保存されたデータを SELECT で取得する流れを考える。

▼ SELECT 文の例

```
SELECT name
FROM users
WHERE age >= 20;
```

SELECT 文も、Tokenizer と SimpleParser によって解析され、Statement.Select へ変換される。

```
Statement.Select
取得列
  name
対象テーブル
  users
条件
  age >= 20
```

QueryExecutor は、Catalog から users テーブルを取得し、Table.scan() を呼び出す。

Table は users.tbl をページ単位で読み込み、各スロットの使用フラグを確認する。使用フラグが 1 であれば、そのスロットには有効なレコードが保存されている。

ただし、ファイルに保存されているのは Row ではなくバイト列である。そのため Table は、Schema を参照しながらバイト列を Row へ戻す。

```
users.tblのバイト列
↓
Schemaを参照
id    : INTEGER
```

第6章 テーブルが定義され、データが保存されるまで

```
name : STRING(20)
age  : INTEGER
```

↓
値を読み取る

```
4バイト → id
4バイト → nameの長さ
20バイト → name
4バイト → age
```

↓
Rowへ復元

復元された Row は、QueryExecutor へ返される。

QueryExecutor は、WHERE 条件である `age >= 20` を評価し、条件に一致する Row だけを残す。その後、SELECT で指定された `name` 列だけを取り出し、結果として表示する。

SELECT文

↓
Statement.Select

↓
Table.scan()

↓
バイト列をRowへ復元

↓
WHERE条件で絞り込む

↓
name列だけ取り出す

↓
画面へ表示

ここでも Schema が重要な役割を持つ。Schema がなければ、ファイルのどの位置にどの列が保存されているか分からない。

たとえば、レコードの先頭から 4 バイトを読んだとしても、それが `id` なのか `age` なのかは、Schema を見なければ判断できない。

6.3 UPDATE では RecordId を使って同じ位置を書き換える

UPDATE では、条件に一致する行を探し、その行の内容を書き換える。

▼ UPDATE 文の例

```
UPDATE users
SET age = 30
WHERE id = 1;
```

Parser は、この SQL を次のような `Statement.Update` へ変換する。

```
Statement.Update
対象テーブル
  users
更新する列
  age
新しい値
  30
条件
  id = 1
```

QueryExecutor は、Table から Row を読み取り、`id = 1` に一致する行を探す。

ただし、UPDATE では Row の内容だけでなく、その Row がファイル上のどこに保存されているかも必要になる。ファイル上の位置が分からなければ、どの部分を書き換えればよいか判断できないからである。

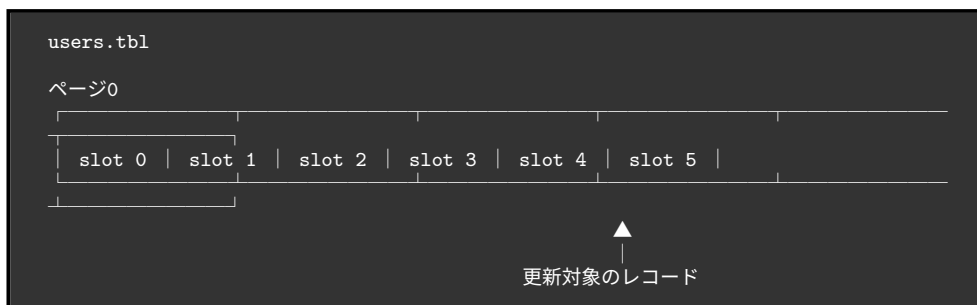
そこで Table は、Row とともに `RecordId` を返す。

`RecordId` は、ページ番号とスロット番号を持つ。

```
RecordId
page = 0
slot = 5
```

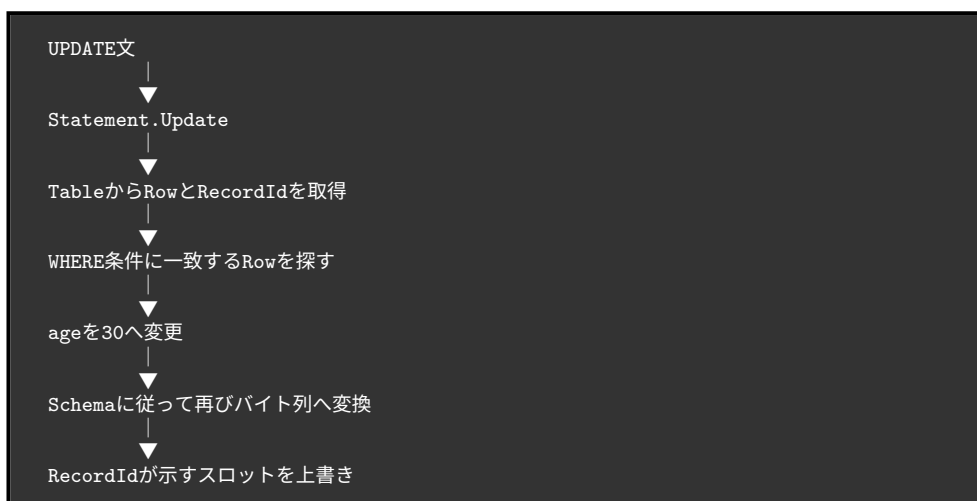
これは、対象レコードがページ 0 のスロット 5 に保存されていることを表す。

第6章 テーブルが定義され、データが保存されるまで



QueryExecutor は、対象 Row の age を 30 へ変更する。その後、変更後の Row と RecordId を Table.update() へ渡す。

Table は RecordId からファイル内の位置を計算し、該当するスロットだけを新しいバイト列で上書きする。



UPDATE では、テーブルファイル全体を書き直す必要はない。RecordId によって対象の位置が分かるため、そのスロットだけを直接更新できる。

6.4 DELETE ではレコードを物理的に消去しない

DELETE では、条件に一致する行を削除する。

▼ DELETE 文の例

6.4 DELETE ではレコードを物理的に消去しない

```
DELETE FROM users
WHERE id = 1;
```

Parser は、この SQL を次のような `Statement.Delete` へ変換する。

```
Statement.Delete

対象テーブル
    users

条件
    id = 1
```

QueryExecutor は UPDATE と同様に、Table から Row と RecordId を取得し、条件に一致する行を探す。

対象レコードが見つかったら、その RecordId を `Table.delete()` へ渡す。

ただし、chapter05 の DELETE は、レコード全体をファイルから物理的に消去するわけではない。また、後ろにあるレコードを前へ移動して空きを詰める処理も行わない。

Table は、対象スロットの先頭にある使用フラグを 1 から 0 へ変更する。

削除前

フラグ 1	id=1, name=Taro, age=20
-------	-------------------------



削除後

フラグ 0	id=1, name=Taro, age=20
-------	-------------------------

データ本体のバイト列が残っていたとしても、使用フラグが 0 であれば、そのスロットは未使用として扱われる。

SELECT でスキャンするときも、使用フラグが 0 のスロットは読み飛ばされる。

また、次回 INSERT を行ったとき、そのスロットを新しいレコードの保存先として再利用できる。

第6章 テーブルが定義され、データが保存されるまで



この方式により、DELETE のたびにファイル全体を再構成する必要がなくなる。

6.5 chapter05 におけるテーブルのライフサイクル

ここまでの処理をまとめると、chapter05 のテーブルは、CREATE TABLE によって定義され、その後の SQL によって利用される。

最初に CREATE TABLE が実行され、Schema が作られる。Schema は Catalog へ登録され、catalog.txt へ保存される。同時に、データを保存するための.tbl ファイルが用意される。

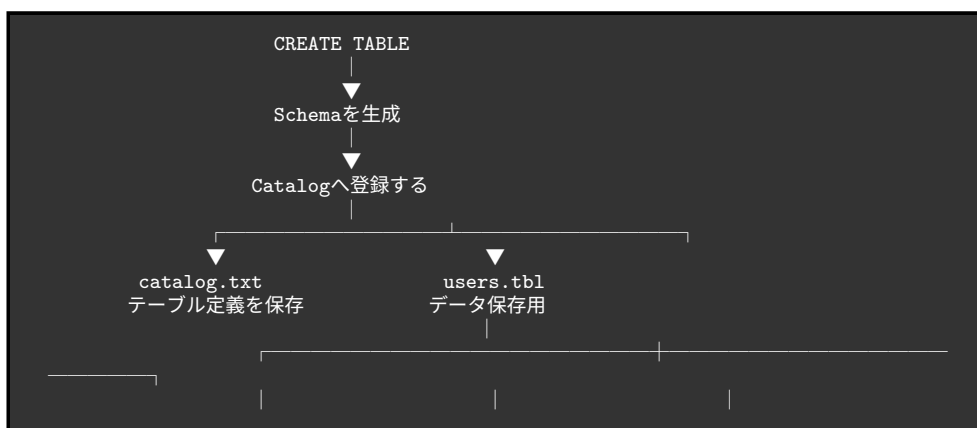
その後、INSERT では Schema を参照しながら値を型変換し、Row をページへ保存する。

SELECT では、Schema を使ってバイト列を Row へ復元し、条件に一致するデータを取出す。

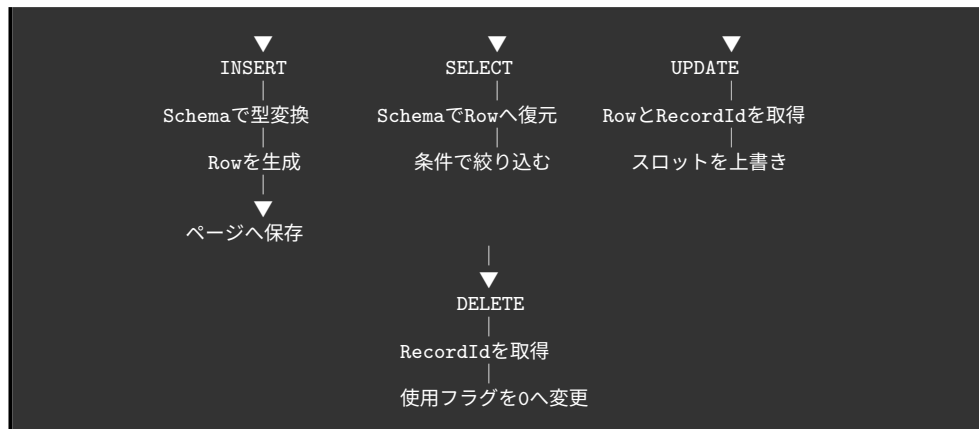
UPDATE では、Row と RecordId を使って対象スロットを上書きする。

DELETE では、対象スロットの使用フラグを未使用へ変更する。

全体の流れは、次のように表せる。



6.5 chapter05 におけるテーブルのライフサイクル



chapter05 では、SQL ごとに処理内容は異なるものの、すべてが同じテーブル定義を共有している。

CREATE TABLE によって生まれた Schema が、その後の INSERT、SELECT、UPDATE、DELETE を支えているのである。

Schema は単なる列名の一覧ではない。値をどの型へ変換するか、レコードを何バイトで保存するか、一ページに何件入るか、ファイルからどの順番で値を読み戻すかを決定する。

Catalog は、その Schema と Table をテーブル名に対応付けて管理する。

QueryExecutor は、Parser が作った Statement を受け取り、Catalog から必要な Schema と Table を取り出す。

Table は、Schema をもとに Row とバイト列を相互に変換し、ページファイル进行操作する。

この役割分担によって、chapter05 では、SQL の解析、テーブル定義、データ操作、ファイル保存が明確に分離されている。

ユーザーが入力した一つの SQL は、Parser によって意味を読み取られ、Statement として整理され、QueryExecutor によって実行内容へ変換される。その後、Catalog、Schema、Table、Row を経由し、最終的にページファイルへ到達する。



第6章 テーブルが定義され、データが保存されるまで



chapter05 の設計を理解するうえで重要なのは、この流れを個別のクラスの集まりとして見るのではなく、一つの連続した処理として捉えることである。

Parser だけではデータは保存されない。Schema だけでも SQL は実行できない。Table だけでは、どのテーブルを操作するか判断できない。

それぞれのクラスが担当する情報を次のクラスへ渡すことで、SQL 文字列が最終的なファイル操作へ変換されているのである。

第 7 章

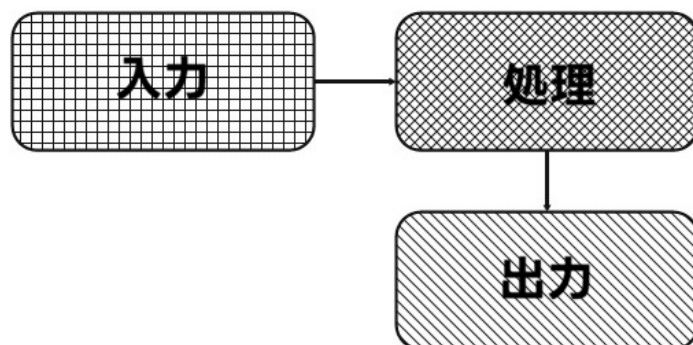
章タイトル

ここからすべてが始まる！

7.1 節のキャプション

画像は articles フォルダの下の images フォルダに入れて、こんな風に指定すると表示されます。(図 7.1)

Re:VIEW サンプル図



▲図 7.1 画像はここに表示される

第7章 章タイトル

注釈を書くこともできます。^{*1}

Re:VIEW の文法について詳しくは、Re:VIEW フォーマットガイドを参照してください。

- Re:VIEW フォーマットガイド
 - <https://github.com/kmuto/review/blob/master/doc/format.ja.md>

^{*1} 注釈はこんな風に表示される

第 8 章

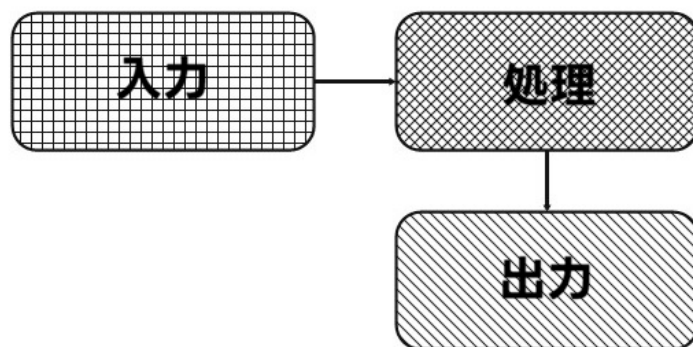
章タイトル

ここからすべてが始まる！

8.1 節のキャプション

画像は articles フォルダの下の images フォルダに入れて、こんな風に指定すると表示されます。(図 8.1)

Re:VIEW サンプル図



▲図 8.1 画像はここに表示される

第8章 章タイトル

注釈を書くこともできます。^{*1}

Re:VIEW の文法について詳しくは、Re:VIEW フォーマットガイドを参照してください。

- Re:VIEW フォーマットガイド
 - <https://github.com/kmuto/review/blob/master/doc/format.ja.md>

^{*1} 注釈はこんな風に表示される

著者紹介

第1章 ひつじ / @mhidaka

ひつじだよ～

Re:VIEW テンプレート

2026 年 4 月 11 日 技術書典 20 版 v1.0.0

著 者 TechBooster 編

編 集 mhidaka

発行所 TechBooster

(C) 2017-2026 TechBooster